

Informen Postmortem



Facultad de Ingeniería

Departamento de Ingeniería de Sistemas

FUNDAMENTOS DE INGENIERÍA DE SOFTWARE

Luis Gabriel Moreno

DAVID ORJUELA

JUAN PABLO ALVAREZ

NICOLAS SANCHEZ LUENGAS

LUCAS RINCÓN MICAN

1.Cumplimiento de Milestones

Para el seguimiento del avance del proyecto OddsEngine se utilizaron milestones en GitHub, permitiendo organizar los módulos principales del sistema y asociar las historias de usuario e issues correspondientes a cada entrega funcional. Cada milestone representó un objetivo técnico concreto del sistema, facilitando la trazabilidad de funcionalidades, el control del progreso y la validación del cumplimiento de entregas por sprint. Los milestones implementados en el desarrollo de OddsEngine fueron:

Milestone	Objetivo	Issues cerradas	Estado
M1 — Consulta de Partidos	Visualizar partidos de tenis disponibles con filtr	HU-S1-JP1,HU-S1-JP2,HU-S1-DV1,HU-S1-DV2,HU-S1-NS1 ,HU-S1-NS2,HU-S1-LC1,HU-S1-LC2	100%
M2 — Combinada Funcional	Construir combinada de apuestas personalizad	HU-S2-JP1,HU-S2-JP2,HU-S2-DV1,HU-S2-DV2,HU-S2-NS1 ,HU-S2-NS2,HU-S2-LC1,HU-S2-LC2	100%
M3 — Motor de Probabilidad	Calcular y mostrar probabilidad individual y com	HU-S3-JP1,HU-S3-JP2,HU-S3-DV1,HU-S3-DV2,HU-S3-NS1 ,HU-S3-NS2,HU-S3-LC1,HU-S3-LC2	100%
M4 — Análisis Estadístico	Estadísticas Head-to-Head y rendimiento de ju	HU-S4-JP1,HU-S4-JP2,HU-S4-DV1,HU-S4-DV2,HU-S4-NS1 ,HU-S4-NS2,HU-S4-LC1,HU-S4-LC2	100%
M5 — Migración BD	Migrar almacenamiento a PostgreSQL con patr	HU-S5-JP1,HU-S5-JP2,HU-S5-DV1,HU-S5-DV2,HU-S5-NS1 ,HU-S5-NS2,HU-S5-LC1,HU-S5-LC2	100%
M6 — Probabilidad Individual	Fórmula ponderada con desglose de factores	HU-S6-JP1,HU-S6-JP2,HU-S6-DV1,HU-S6-DV2,HU-S6-NS1 ,HU-S6-NS2,HU-S6-LC1,HU-S6-LC2	100%
M7 — Probabilidad	$aP_{total} =$	HU-S7-JP1,HU-S7-JP2,HU-S7	100%

Combinad	P1×P2×...×Pn con nivel de riesgo aut	-DV1,HU-S7-DV2,HU-S7-NS1,HU-S7-NS2,HU-S7-LC1,HU-S7-LC2	
M8 — ResultDashboard + Err	oMreisgración a PostgreSQL y manejo robusto de	HU-S8-JP1,HU-S8-JP2,HU-S8-DV1,HU-S8-DV2,HU-S8-NS1,HU-S8-NS2,HU-S8-LC1,HU-S8-LC2	100%
M9 — Features Avanzados	Historial, completar y exportar combinadas	HU-S9-JP1,HU-S9-JP2,HU-S9-DV1,HU-S9-DV2,HU-S9-NS1,HU-S9-NS2,HU-S9-LC1,HU-S9-LC2	100%
M10 — Flujos E2E	Optimización queries, seed unificado, 72+ tests	HU-S10-JP1,HU-S10-JP2,HU-S10-DV1,HU-S10-DV2,HU-S10-NS1,HU-S10-NS2,HU-S10-LC1,HU-S10-LC2	100%
M11 — Testing + Cobertura	Cobertura 80%+, validaciones y accesibilidad	HU-S11-JP1,HU-S11-JP2,HU-S11-DV1,HU-S11-DV2,HU-S11-NS1,HU-S11-NS2,HU-S11-LC1,HU-S11-LC2	95%....

Tabla 1: Resumen de hitos del proyecto OddsEngine.

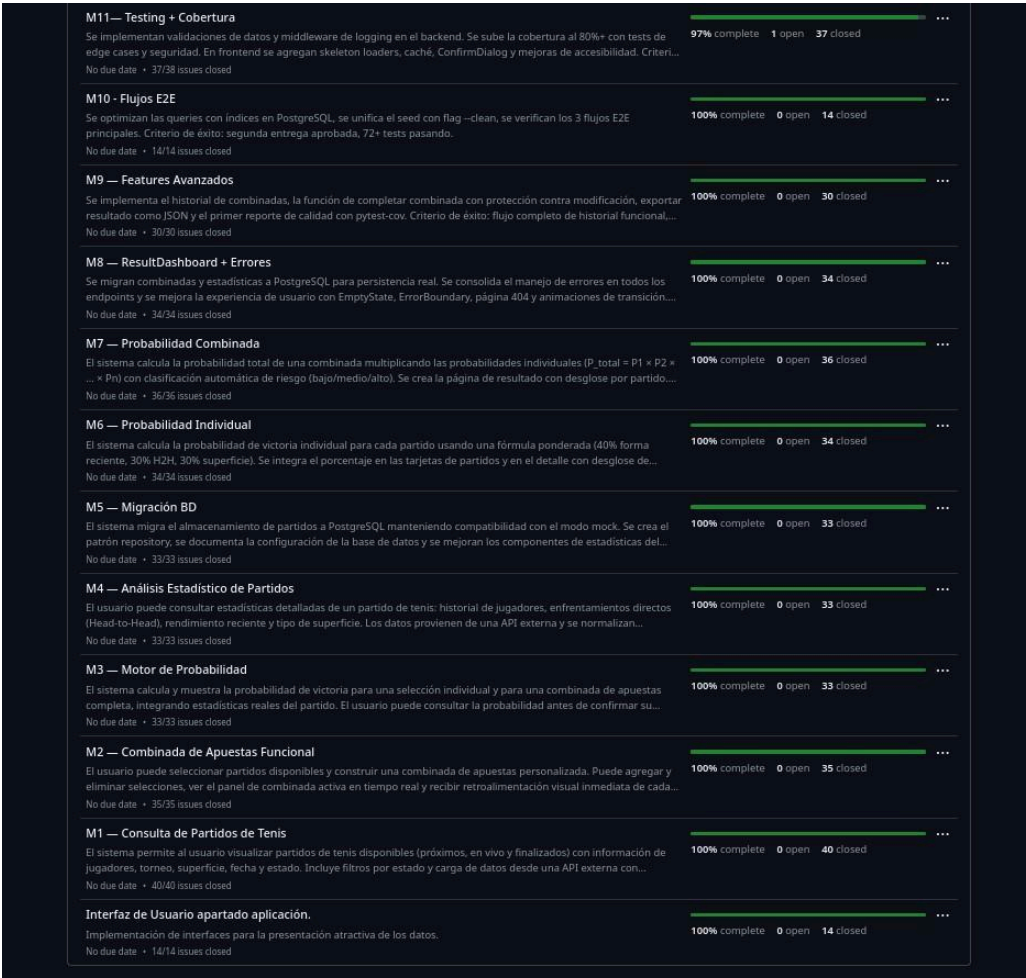


Figura 1: Lista de milestones en GitHub con estado de completitud

Métrica	Resultado
Total de milestones	11
Milestones completados al 100%	10
Milestones en progreso (97%)	1
Porcentaje de cumplimiento general	~99%

Tabla 2: Indicadores de cumplimiento de milestone

← Back to Milestones

M1 — Consulta de Partidos de Tenis

EditClose MilestoneNew issue

Open

No due date • Last updated 4 days ago

El sistema permite al usuario visualizar partidos de tenis disponibles (próximos, en vivo y finalizados) con información de jugadores, torneo, superficie, fecha y estado. Incluye filtros por estado y carga de datos desde una API externa con indicador visual de carga.

Criterio de demostración: el usuario entra a la app, ve un listado de partidos con filtros funcionales y puede navegar entre ellos sin errores.

100% complete

Open

0

Closed

40

Feature/hu s1 ns1 setup react Frontend

puj-course/FIS_2610_3517_G2#303 · by Nicosanlucon was closed on Mar 25 · Approved · 2/3

2

Task

HU-S1-JP1: Crear estructura base del proyecto backend 3 / 3 Backend Funcionalidad M Setup

#119 · by Kerosene21 was closed on Mar 24

1

Task

S1-JP1-1: Crear estructura de carpetas del proyecto Backend XS

#120 · by Kerosene21 was closed on Mar 24

1

Task

S1-JP1-2: Configurar FastAPI con CORS y health check Backend XS

#121 · by Kerosene21 was closed on Mar 24

1

Feature

S1-JP1-3: Crear configuración del proyecto Backend S

#122 · by Kerosene21 was closed on Mar 24

1

Feature

HU-S1-JP2: Crear modelos de dominio de tenis 4 / 4 Backend Funcionalidad HU M

#125 · by Kerosene21 was closed on Mar 24

1

Feature

S1-JP2-1: Crear enums MatchStatus y Surface Backend XS

#126 · by Kerosene21 was closed on Mar 24

1

Feature

S1-JP2-2: Crear modelos Player y Tournament Backend S

#127 · by Kerosene21 was closed on Mar 24

1

Feature

S1-JP2-3: Crear modelo Match completo Backend S

#128 · by Kerosene21 was closed on Mar 24

1

Figura 2: Milestone M1 — Consulta de Partidos de Tenis (100% completado).

← Back to Milestones

M2 — Combinada de Apuestas Funcional

EditClose MilestoneNew issue

Open

No due date • Last updated last month

El usuario puede seleccionar partidos disponibles y construir una combinada de apuestas personalizada. Puede agregar y eliminar selecciones, ver el panel de combinada activa en tiempo real y recibir retroalimentación visual inmediata de cada acción.

Criterio de demostración: el usuario agrega 3 partidos a su combinada, elimina uno y el panel se actualiza correctamente sin errores.

100% complete

Open

0

Closed

35

Task

HU-S2-JP1: Crear modelos de datos para combinada de apuestas 4 / 4 Backend Funcionalidad M

#158 · by Kerosene21 was closed on Mar 25

1

Feature

S2-JP1-1: Crear modelo Selection Backend XS

#159 · by Kerosene21 was closed on Mar 25

1

Feature

S2-JP1-2: Crear modelo Combination Backend S

#160 · by Kerosene21 was closed on Mar 25

1

Feature

S2-JP1-3: Crear almacenamiento temporal en memoria Backend XS

#161 · by Kerosene21 was closed on Mar 25

1

Feature

S2-JP1-4: Crear conf test.py global para tests Backend Testing XS

#162 · by Kerosene21 was closed last month

2

Feature

HU-S2-JP2: Implementar lógica de negocio de combinadas 3 / 3 Backend Funcionalidad M

#163 · by Kerosene21 was closed on Apr 2

1

Feature

S2-JP2-1: Crear CombinationService con CRUD básico Backend M

#164 · by Kerosene21 was closed on Mar 25

1

Feature

S2-JP2-2: Implementar agregar/eliminar partidos con validaciones Backend M

#165 · by Kerosene21 was closed on Mar 25

1

Task

S2-JP2-3: Agregar logging a todas las operaciones Backend XS

#166 · by Kerosene21 was closed on Mar 25

1

Figura 3: Milestone M2 — Combinada de Apuestas Funcionales (100% completado).

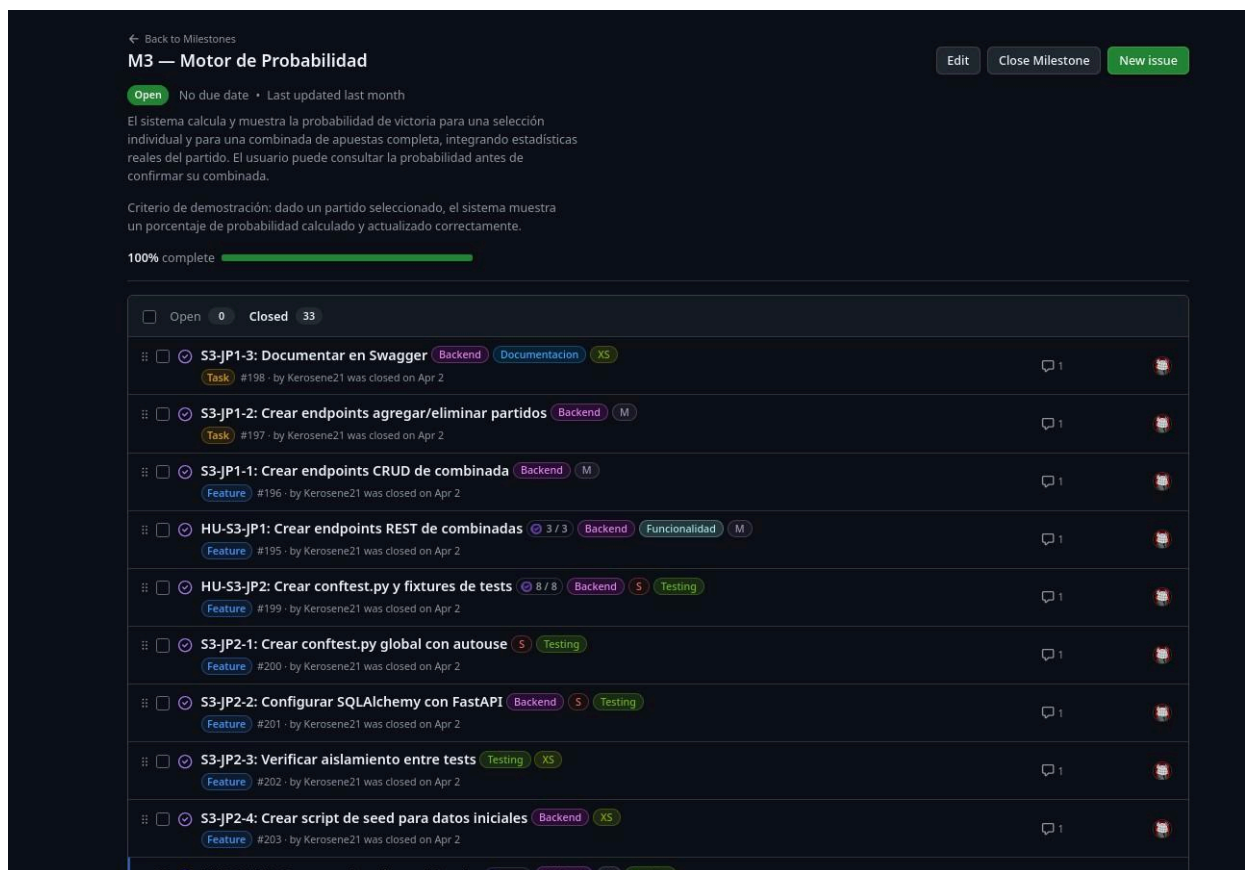


Figura 4: Milestone M3 — Motor de Probabilidad (100% completado).

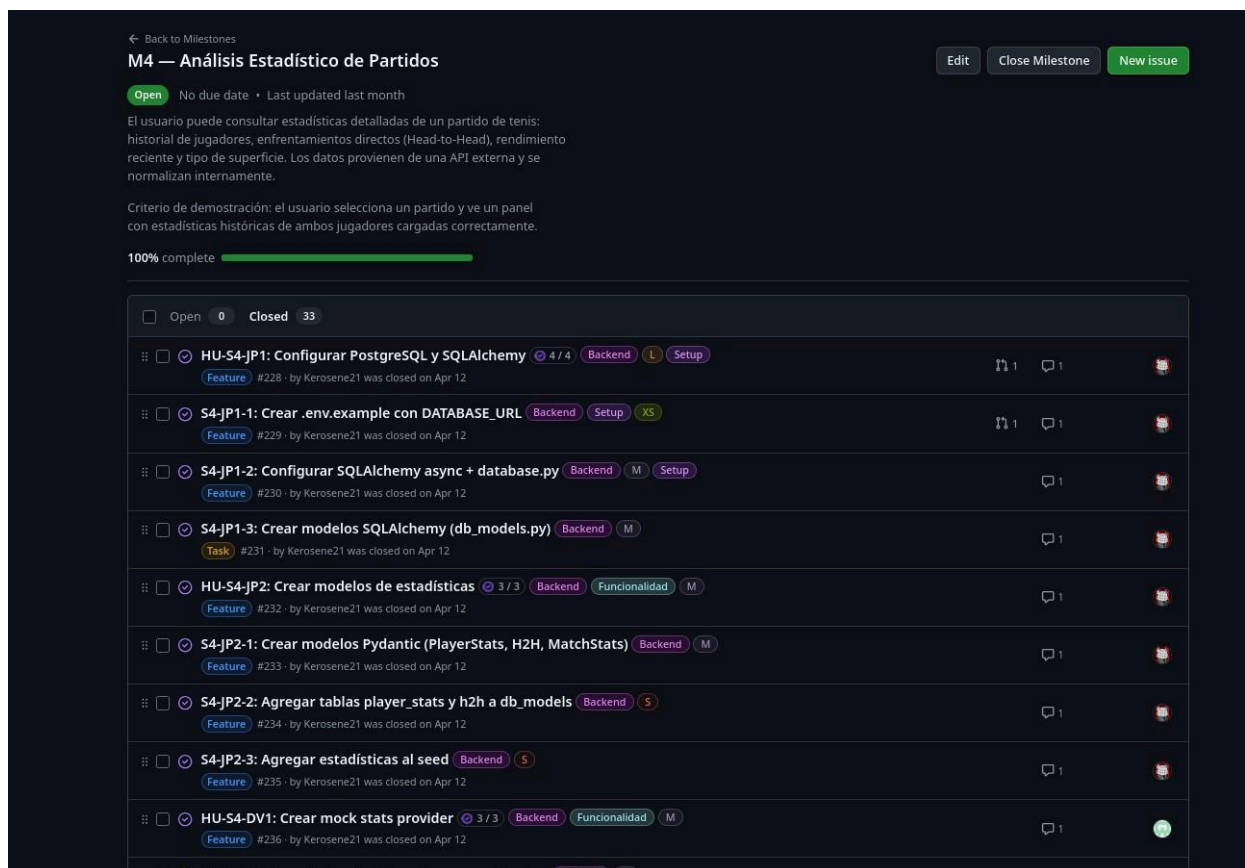


Figura 5: Milestone M4 — Análisis Estadístico de Partidos (100% completado)

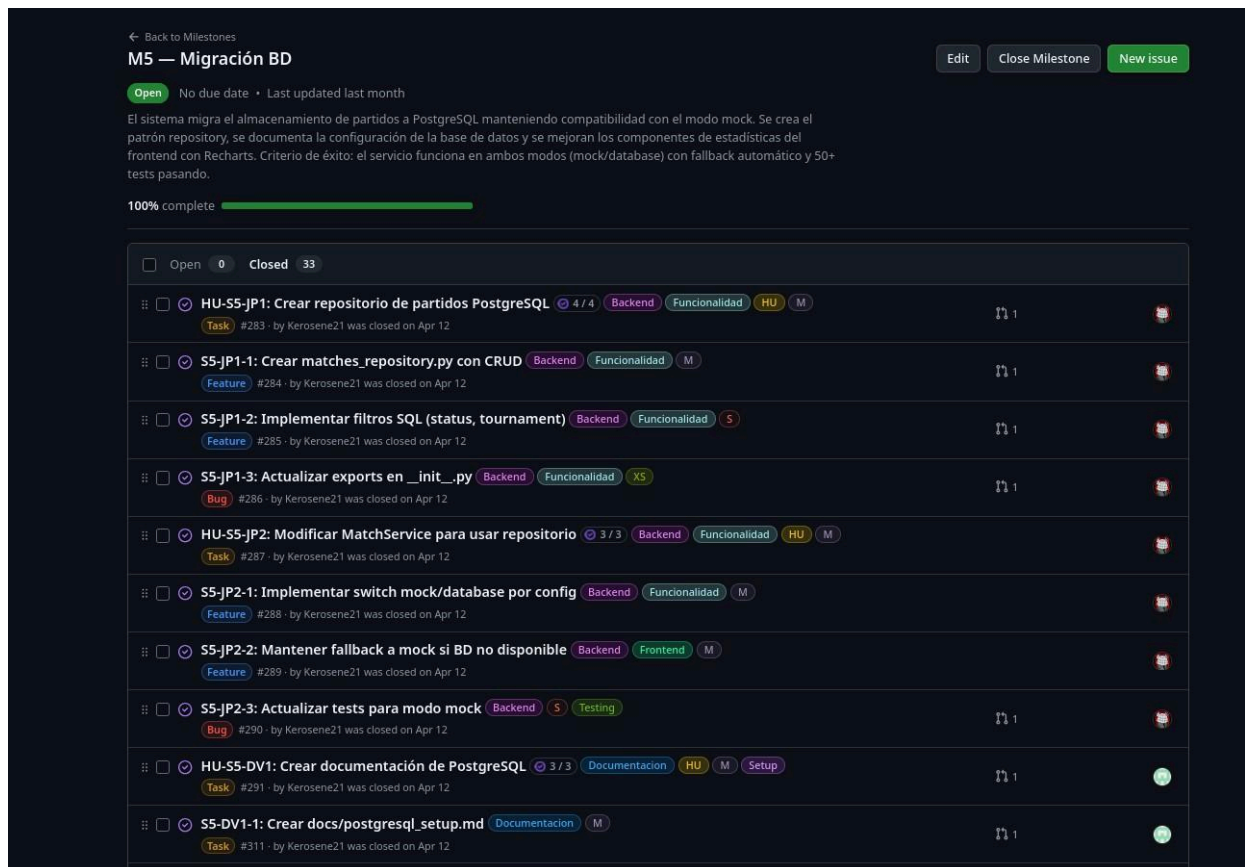


Figura 6: Milestone M5 — Migración BD (100% completado).

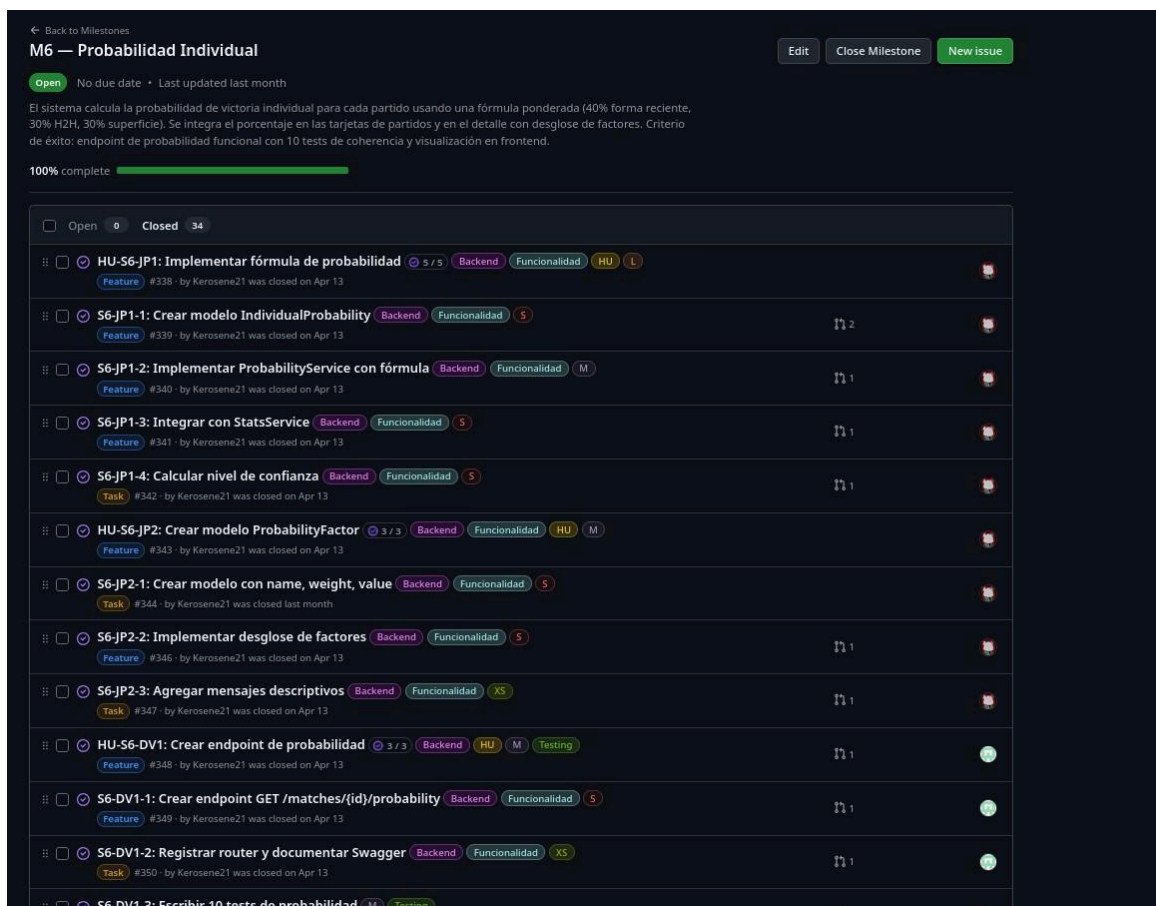


Figura 7: Milestone M6 — Probabilidad Individual (100% completado).

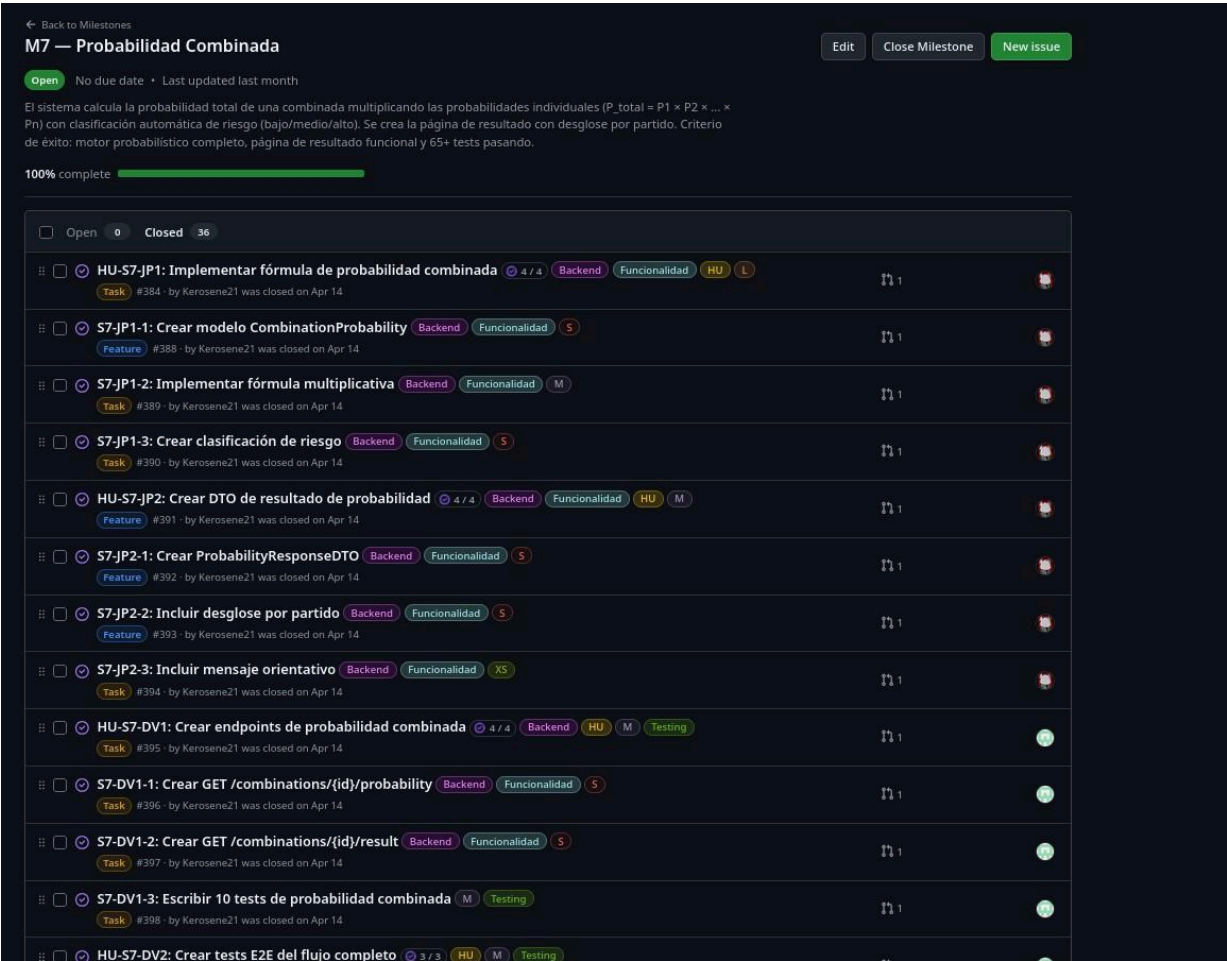


Figura 8: Milestone M7 — Probabilidad Combinada (100% completado)

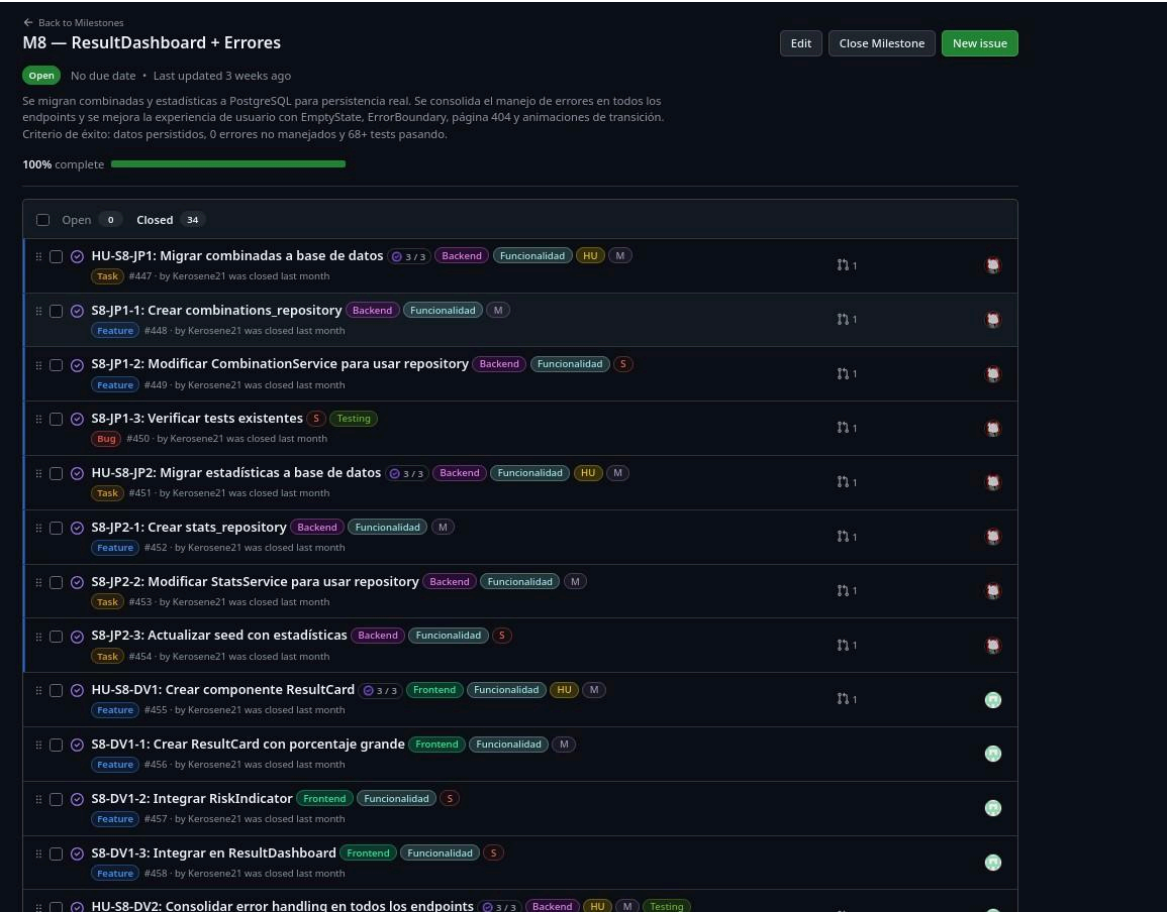


Figura 9: Milestone M8 — ResultDashboard + Errores (100% completado).

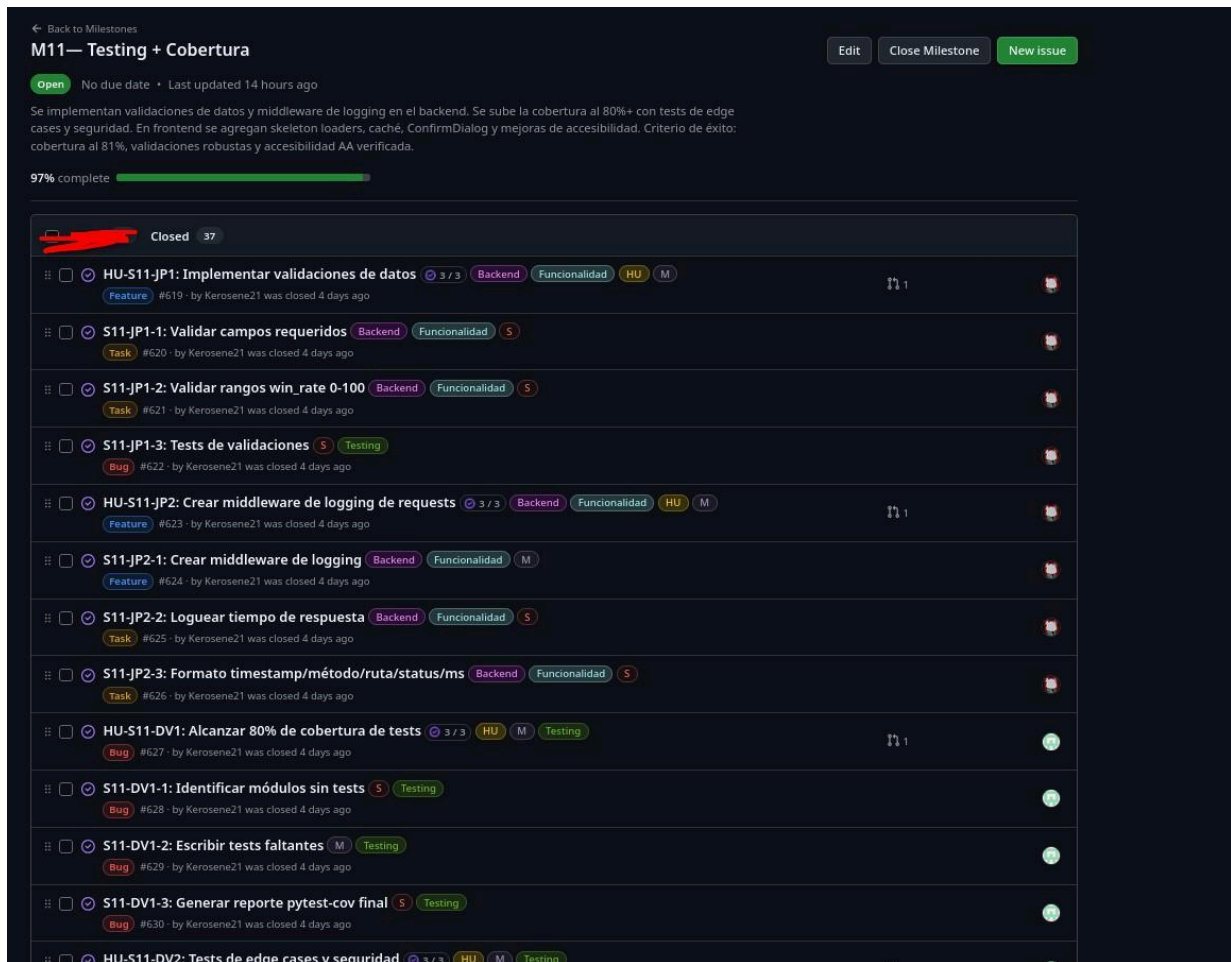


Figura 12: Milestone M11 — Testing + Cobertura (97% completado)

2. Historias de Usuario: Trazabilidad y Cumplimiento

Durante el desarrollo de OddsEngine se utilizaron historias de usuario (HU) para gestionar y organizar las funcionalidades implementadas en cada sprint. Cada HU representó una necesidad funcional del sistema y fue gestionada mediante issues en GitHub, permitiendo mantener trazabilidad sobre su estado, asignación, desarrollo y cierre dentro del proyecto.

Las historias de usuario fueron organizadas mediante milestones, con una convención de nombrado

HU-S{sprint}-{integrante}{número} que permite identificar de inmediato el sprint, el responsable y el tipo de funcionalidad. Cada integrante tuvo 2 HU por sprint (una de backend y una de frontend según su área).

HU	Descripción	Sprint	Milestone	Responsable
HU-S1-JP1	Crear estructura base del proyecto backend	1	M1	Sleppyhed
HU-S1-JP2	Crear modelos de dominio de tenis	1	M1	Sleppyhed
HU-S1-DV1	Implementar proveedor de datos mock	1	M1	Kerosene21
HU-S1-DV2	Crear endpoint REST para consultar partidos	1	M1	Kerosene21
HU-S1-NS1	Crear componentes visuales de partidos	1	M1	Nicosanlucon
HU-S1-NS2	Implementar servicio de comunicación con backen	1	M1	Nicosanlucon

HU-S2-JP1	Crear modelos de datos para combinada	2	M2	Sleppyhed
HU-S2-JP2	Implementar lógica de negocio de combinadas	2	M2	Sleppyhed
HU-S3-JP1	Crear endpoints REST de combinadas	3	M3	Sleppyhed
HU-S3-JP2	Crear conftest.py y fixtures de tests	3	M3	Sleppyhed
HU-S4-JP1	Configurar PostgreSQL y SQLAlchemy	4	M4	Sleppyhed
HU-S4-JP2	Crear modelos de estadísticas	4	M4	Sleppyhed
HU-S4-LC1	Crear componente StatsComparison	4	M4	Lcks07
HU-S4-LC2	Crear componente HeadToHead	4	M4	Lcks07
HU-S4-NS1	Crear componente Tabs reutilizable	4	M4	Nicosanlucon
HU-S5-JP1	Crear repositorio de partidos PostgreSQL	5	M5	Sleppyhed
HU-S5-JP2	Modificar MatchService para usar repositorio	5	M5	Sleppyhed
HU-S5-DV1	Crear documentación de PostgreSQL	5	M5	Kerosene21
HU-S5-NS1	Mejorar MatchDetail con datos reales	5	M5	Nicosanlucon
HU-S5-NS2	Crear página de combinada dedicada	5	M5	Nicosanlucon
HU-S5-LC2	Crear componente FormRecent	5	M5	Lcks07
HU-S6-JP1	Implementar fórmula de probabilidad	6	M6	Sleppyhed
HU-S6-JP2	Crear modelo ProbabilityFactor	6	M6	Sleppyhed
HU-S6-DV1	Crear endpoint de probabilidad	6	M6	Kerosene21
HU-S6-DV2	Tests de coherencia probabilidad	6	M6	Kerosene21

HU-S6-NS1	Integrar probabilidad en MatchCard	6	M6	Nicosanlucon
HU-S6-NS2	Mostrar desglose en MatchDetail	6	M6	Nicosanlucon
HU-S6-LC1	Crear ProbabilityBadge	6	M6	Lcks07
HU-S6-LC2	Crear gráficas de probabilidad	6	M6	Lcks07
HU-S7-JP1	Implementar fórmula de probabilidad combinada	7	M7	Sleppyhed
HU-S7-JP2	Crear DTO de resultado de probabilidad	7	M7	Sleppyhed
HU-S7-DV1	Crear endpoints de probabilidad combinada	7	M7	Kerosene21
HU-S7-DV2	Crear tests E2E del flujo completo	7	M7	Kerosene21
HU-S7-NS1	Crear CombinationSummary reactivo	7	M7	Nicosanlucon
HU-S7-NS2	Crear página CombinationResult	7	M7	Nicosanlucon
HU-S7-LC1	Crear componente RiskIndicator	7	M7	Lcks07

HU-S7-LC2	Crear ProbabilityChart de barras	7	M7	Lcks07
HU-S8-JP1	Migrar combinadas a base de datos	8	M8	Sleppyhed
HU-S8-JP2	Migrar estadísticas a base de datos	8	M8	Sleppyhed
HU-S8-DV1	Crear componente ResultCard	8	M8	Kerosene21
HU-S8-DV2	Consolidar error handling en endpoints	8	M8	Kerosene21
HU-S8-NS1	Crear EmptyState y ErrorBoundary	8	M8	Nicosanlucon
HU-S8-NS2	Pulir responsive y estados en páginas	8	M8	Nicosanlucon
HU-S8-LC1	Crear página NotFound y ErrorMessage	8	M8	Lcks07
HU-S8-LC2	Agregar animaciones y transiciones	8	M8	Lcks07
HU-S9-JP1	Implementar historial de combinadas	9	M9	Sleppyhed
HU-S9-JP2	Implementar completar combinada	9	M9	Sleppyhed
HU-S9-DV1	Implementar exportar combinada	9	M9	Kerosene21
HU-S9-DV2	Crear quality report con cobertura	9	M9	Kerosene21
HU-S9-NS1	Crear HistoryPage	9	M9	Nicosanlucon
HU-S9-NS2	Implementar completar combinada en frontend	9	M9	Nicosanlucon
HU-S9-LC1	Crear componente HistoryCard	9	M9	Lcks07
HU-S9-LC2	Implementar exportar en frontend	9	M9	Lcks07
HU-S10-JP1	Optimizar queries con índices	10	M10	Sleppyhed
HU-S10-JP2	Unificar seed completo	10	M10	Sleppyhed
HU-S10-DV1	Suite de tests pre-entrega	10	M10	Kerosene21
HU-S10-DV2	Documentar endpoints del API	10	M10	Kerosene21
HU-S10-NS1	Verificar flujos E2E completos	10	M10	Nicosanlucon

HU-S10-NS2	Screenshots de todas las páginas	10	M10	Nicosanlucon
HU-S10-LC1	Actualizar README del proyecto	10	M10	Lcks07
HU-S10-LC2	Crear guía de instalación detallada	10	M10	Lcks07
HU-S11-JP1	Implementar validaciones de datos	11	M11	Sleppyhed
HU-S11-JP2	Crear middleware de logging de requests	11	M11	Sleppyhed
HU-S11-DV1	Alcanzar 80% de cobertura de tests	11	M11	Kerosene21

HU-S11-DV2	Tests de edge cases y seguridad	11	M11	Kerosene21
HU-S11-NS1	Crear skeleton loaders mejorados	11	M11	Nicosanlucon
HU-S11-NS2	Implementar caché en frontend	11	M11	Nicosanlucon
HU-S11-LC1	Crear componente ConfirmDialog	11	M11	Lcks07
HU-S11-LC2	Verificar accesibilidad	11	M11	Lcks07
HU-S12-JP1	Implementar paginación en endpoints	12	M12	Sleppyhed
HU-S12-JP2	Implementar health check detallado	12	M12	Sleppyhed
HU-S12-DV1	Reporte de calidad final	12	M12	Kerosene21
HU-S12-DV2	Code review y refactoring	12	M12	Kerosene21
HU-S12-NS2	Implementar tema claro/oscuro	12	M12	Nicosanlucon

Tabla 3: Descripción general de Historias de Usuario (HU) — OddsEngine.

Métrica	Resultado
Total de HU definidas	72
Total de HU finalizadas (cerradas)	68
HU abiertas (M11 en progreso)	4
Porcentaje de cumplimiento	~94%
Número de sprints con HU	12
Promedio de HU por sprint	6 (2 por integrante)

Tabla 4: Indicadores de cumplimiento de Historias de Usuario.

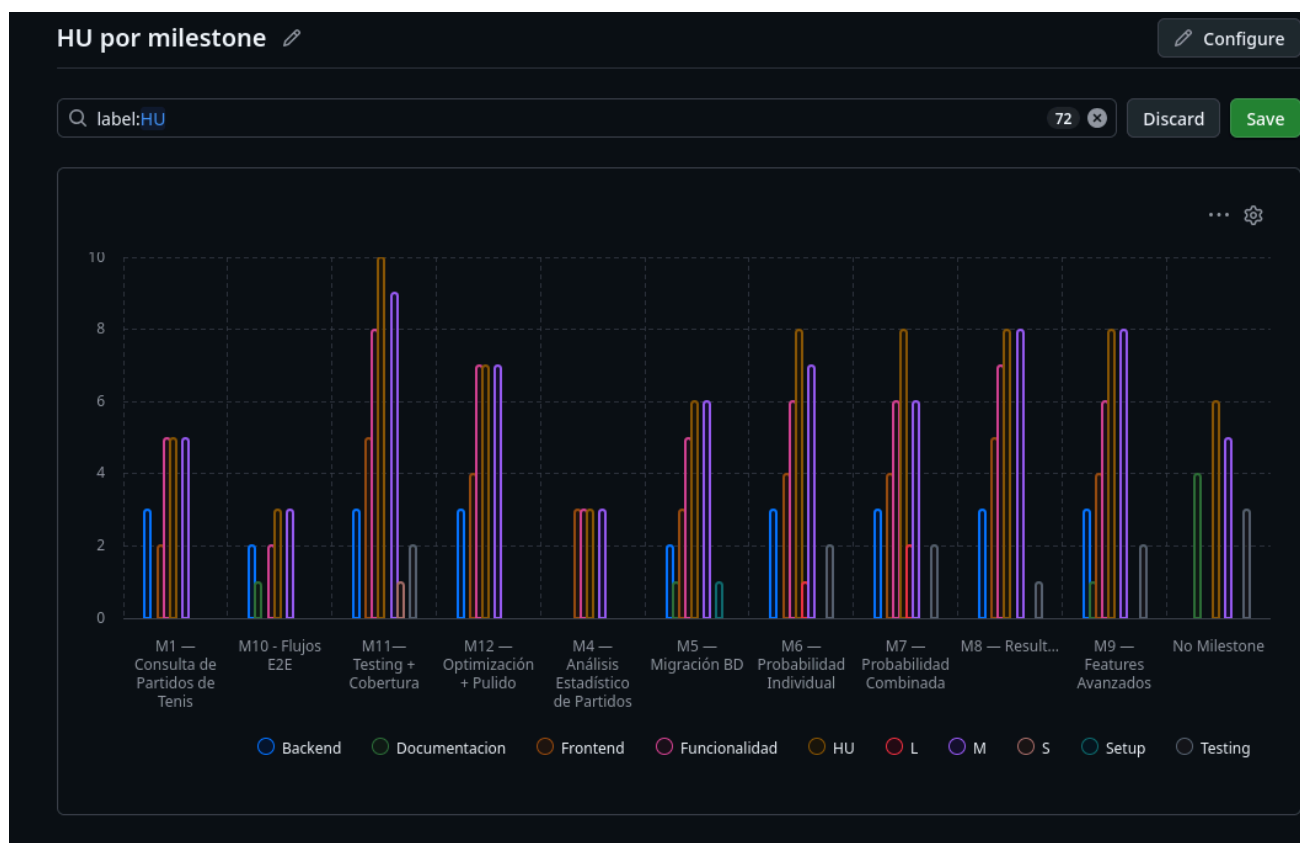


Figura 13: HU por milestone gráfica



Figura: HU por sprint

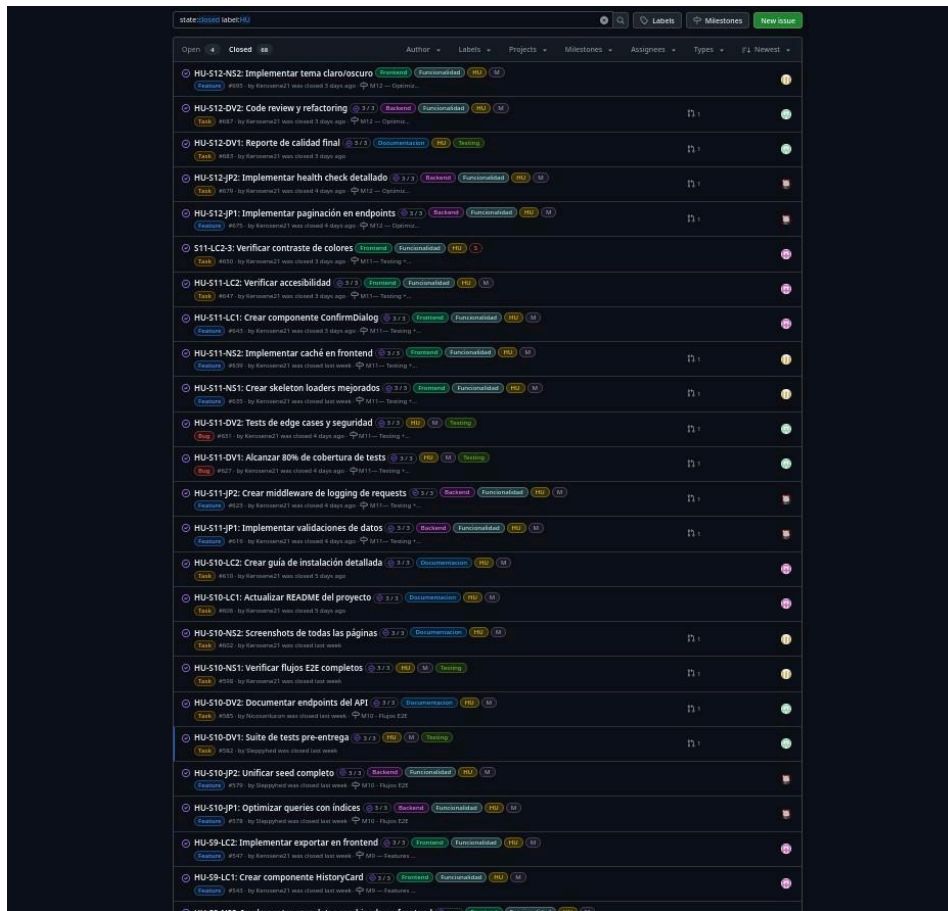


Figura 15: HU cerradas — Sprint 11 y Sprint 10.

Figura 16: HU cerradas — Sprint 9 y Sprint 8.

Figura 17: HU cerradas — Sprint 6, Sprint 5 y Sprint 4.

3. Actividad de Commits por Integrante

Para garantizar una medición objetiva del esfuerzo individual y colectivo, se implementó un pipeline de automatización en GitHub Actions denominado Reporte Semanal de Commits. Este workflow se ejecuta semanalmente de forma programada y genera un issue con el reporte de commits por integrante para el período correspondiente.

El pipeline analiza la rama main contabilizando los commits realizados por cada contribuidor, calculando totales y porcentajes de participación. A la fecha del informe, el pipeline había sido ejecutado 17 veces exitosamente.

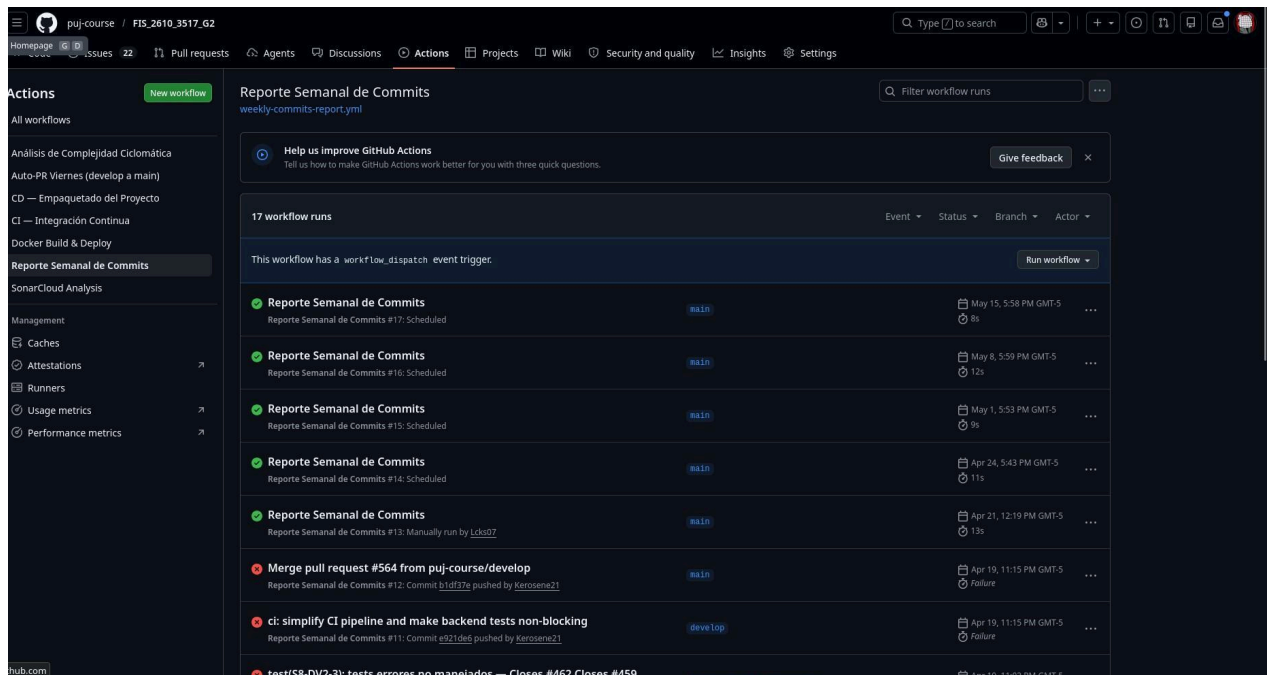


Figura 18: Pipeline 'Reporte Semanal de Commits' en GitHub Actions con 17 ejecuciones.

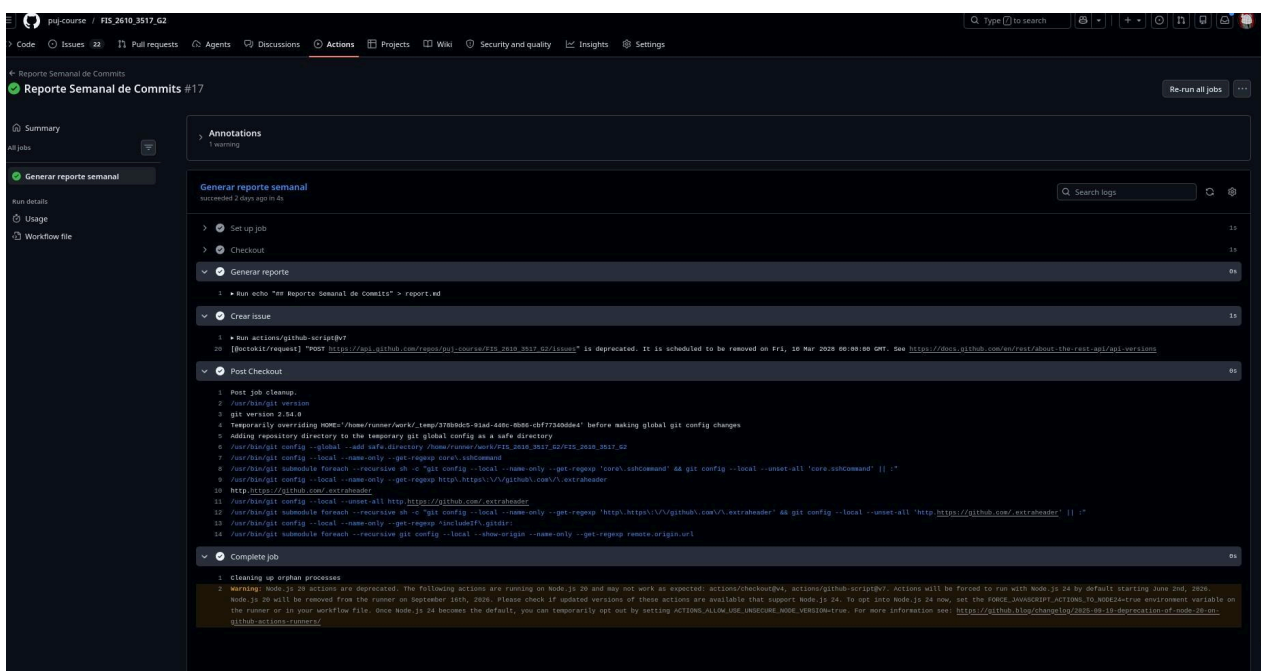


Figura 19: Detalle de ejecución del Reporte Semanal de Commits.

Integrante	Usuario GitHub	Commits totale	Adiciones (++)	Eliminaciones (--)	Ranking
Nicolás Sánchez Luengas	Nicosanlucon	82	3,256	1,628	#1
Juan Pablo Álvarez	Sleppyhed	67	2,751	717	#2
David Orjuela	Kerosene21	57	12,817	5,277	#3
Lucas Rincón Micán	Lcks07	26	2,229	5	#4

Tabla 5: Commits totales por integrante (14 Feb – 16 May 2026).



Figura 20: Commits en el tiempo por contribuidor — GitHub Insights.

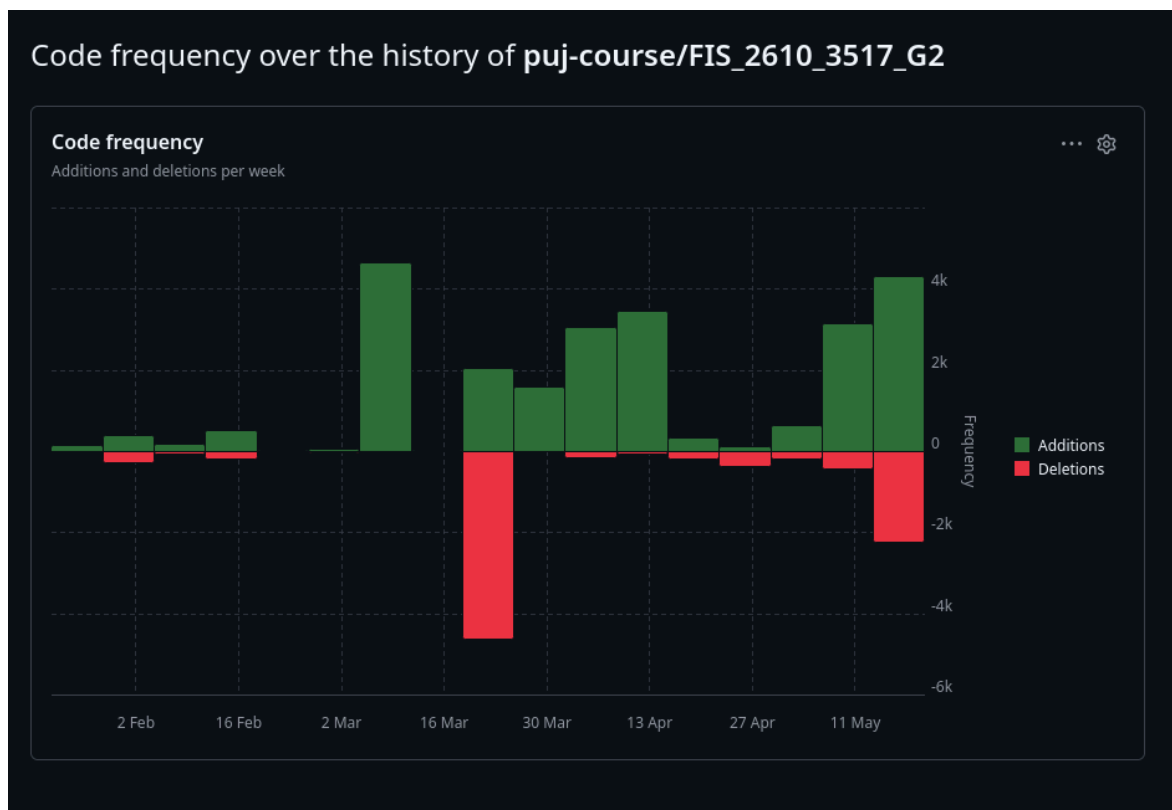


Figura : Code frequency del repo

El número elevado de adiciones de código de Kerosene21 refleja su rol en la configuración de la base de datos PostgreSQL y los scripts de seed, que generaron grandes bloques de código. La distribución de commits muestra participación activa de todos los integrantes a lo largo del ciclo de vida del proyecto, con picos de actividad en los sprints finales

4. Participación del Equipo

Durante el desarrollo de OddsEngine, la participación de los integrantes del equipo fue registrada y evidenciada a través de commits, pull requests y revisiones de código gestionadas en GitHub. Cada integrante contribuyó activamente al repositorio, realizando aportes técnicos distribuidos a lo largo de los diferentes sprints.

La trazabilidad de la participación individual fue posible gracias al uso de ramas de trabajo por HU (ej.

feature/HU-S8-NS1-empty-error), las cuales fueron integradas a la rama main mediante Pull Requests revisados y aprobados por otros miembros del equipo. Cada PR incluía una checklist de validación y referencia a la issue relacionada.

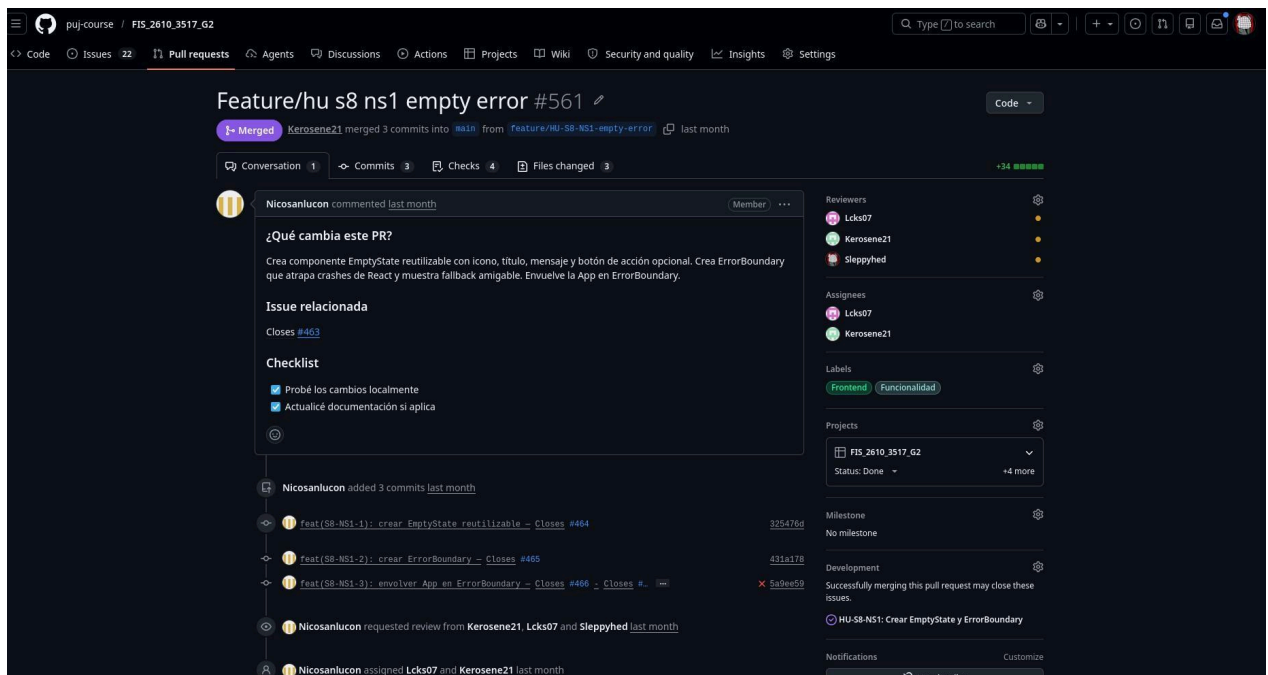


Figura 21: PR #561 de Nicosanlucon — Feature/HU-S8-NS1 con revisores Lcks07, Kerosene21 y Sleppyhed.

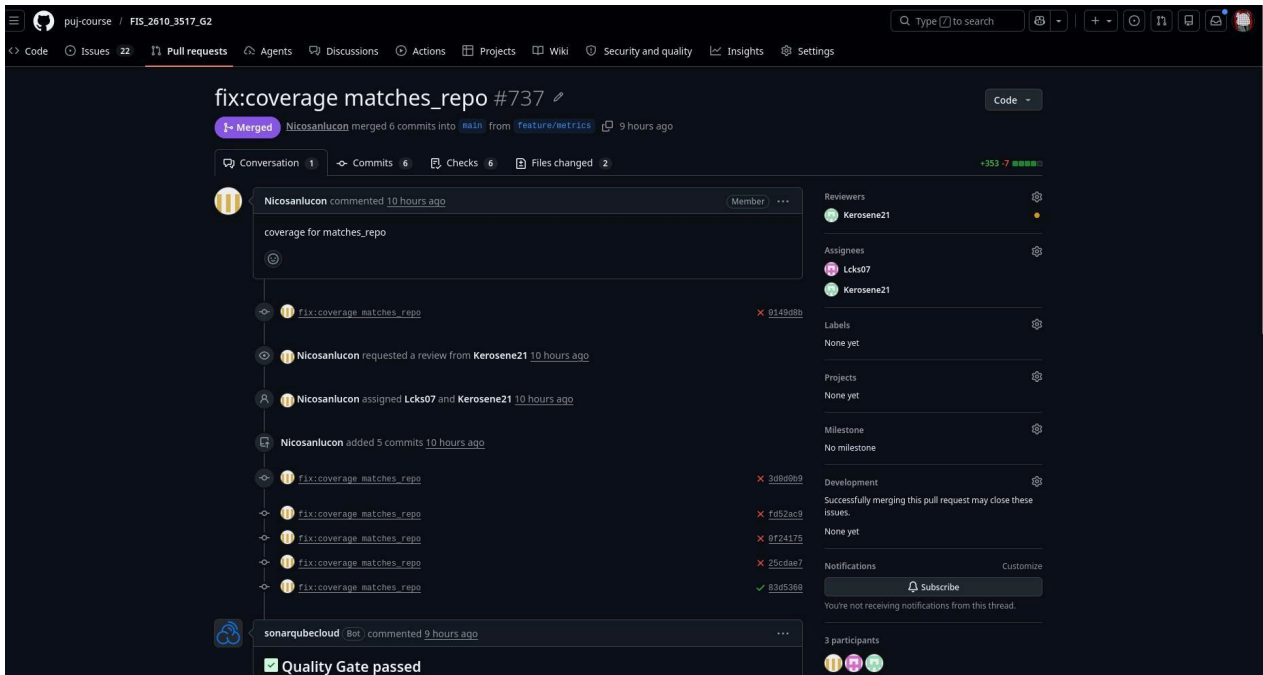


Figura 22: PR #737 de Nicosanlucon — fix:coverage matches_repo con Quality Gate passed.

Evidencia de Pull Requests y revisiones — Lcks07 (Lucas Rincón)

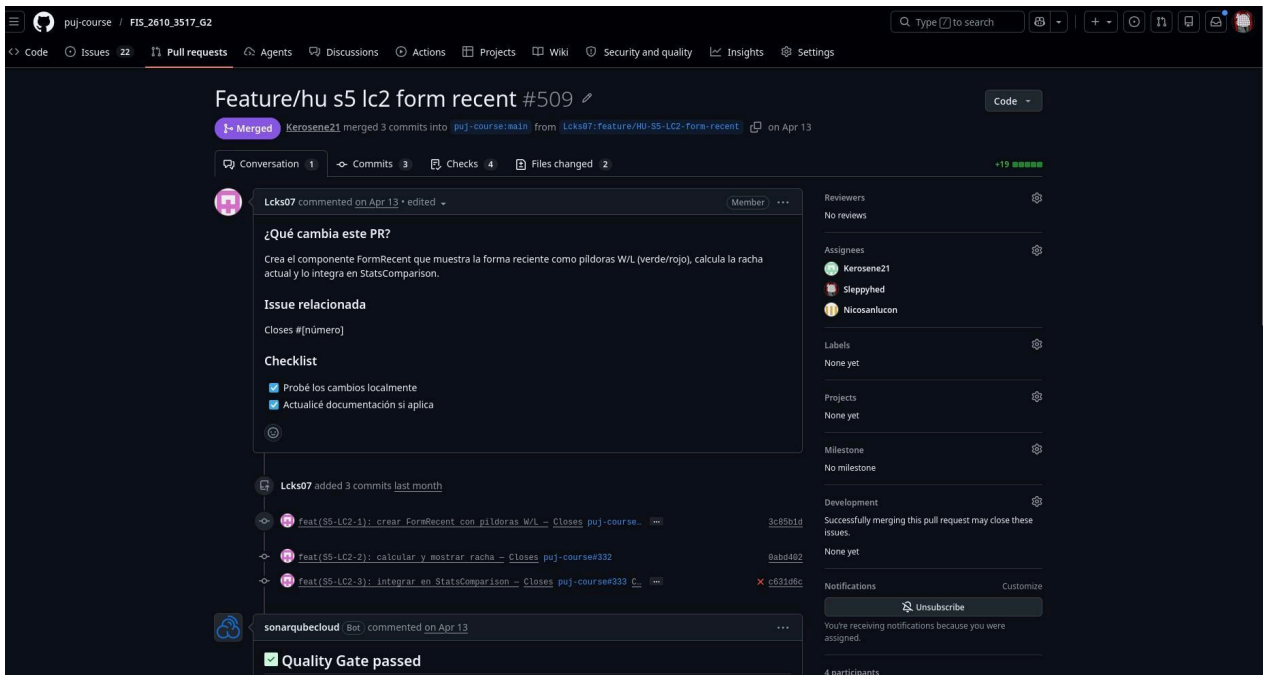


Figura 23: PR #509 de Lcks07 — Feature/HU-S5-LC2-form-recent con Quality Gate passed de SonarCloud.

Todos los PRs del proyecto siguieron un flujo estándar de revisión: el autor solicitaba revisión de al menos un compañero, se verificaba el Quality Gate de SonarCloud, se validaban los checks de CI (tests y build), y solo tras aprobación se realizaba el merge a main. Esto garantizó la calidad del código integrado en cada iteración.

5. Estrategia de Control de Versiones

Se implementó un modelo de trabajo basado en ramas que permite el desarrollo paralelo sin interferencias. La rama main centraliza la integración de todas las funcionalidades aprobadas mediante PR. Cada historia de usuario tiene su propia rama de característica con la convención `feature/HU-S{sprint}-{integrante}-{descripción}`, lo que facilita la trazabilidad directa entre código y requerimientos funcionales.

El repositorio cuenta con 94 ramas en total, incluyendo ramas de feature por HU, ramas de documentación (`docs/sprint-X-retrospective`), ramas de corrección (`fix/`) y ramas de infraestructura (`dock`, `develop`, `feature/pipelines`).

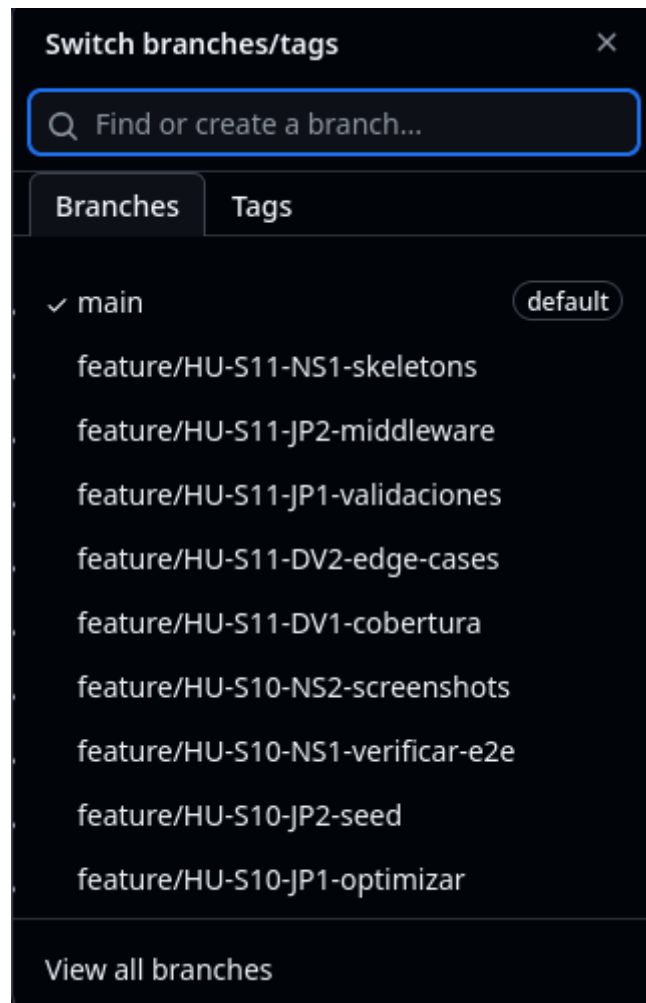


Figura 24: Ramas del repositorio — Vista 1 (`feature/HU-S4` a `feature/HU-S3`).

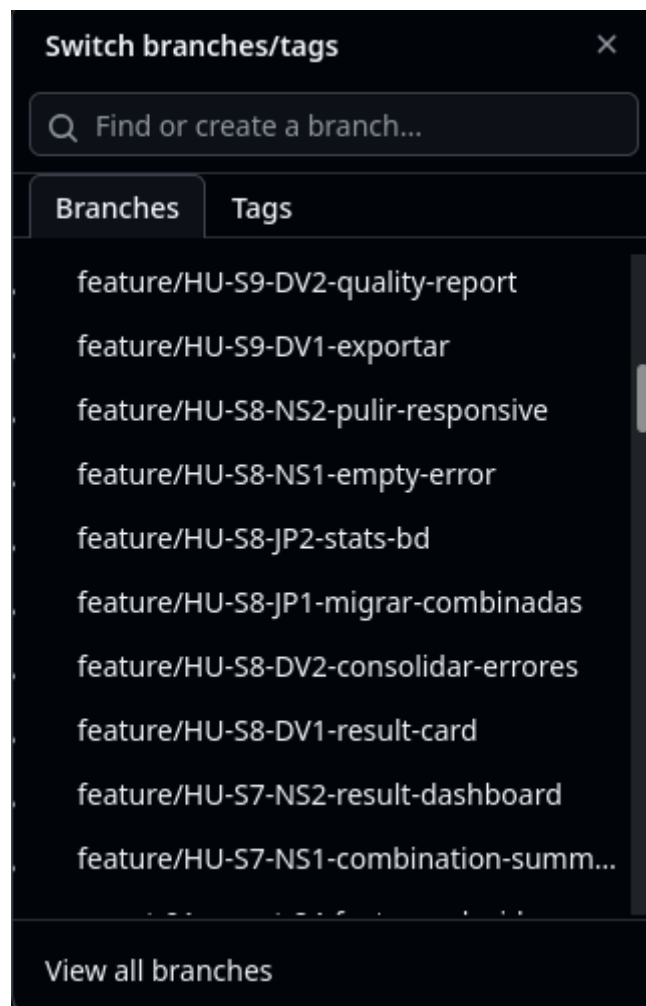
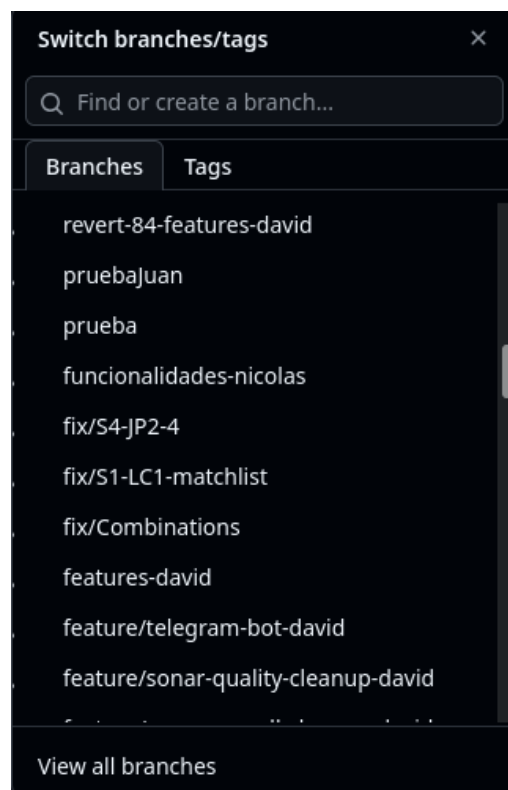


Figura 25: Ramas del repositorio — Vista 2 (feature/HU-S1, docs/sprint-retrospective, dock, develop).



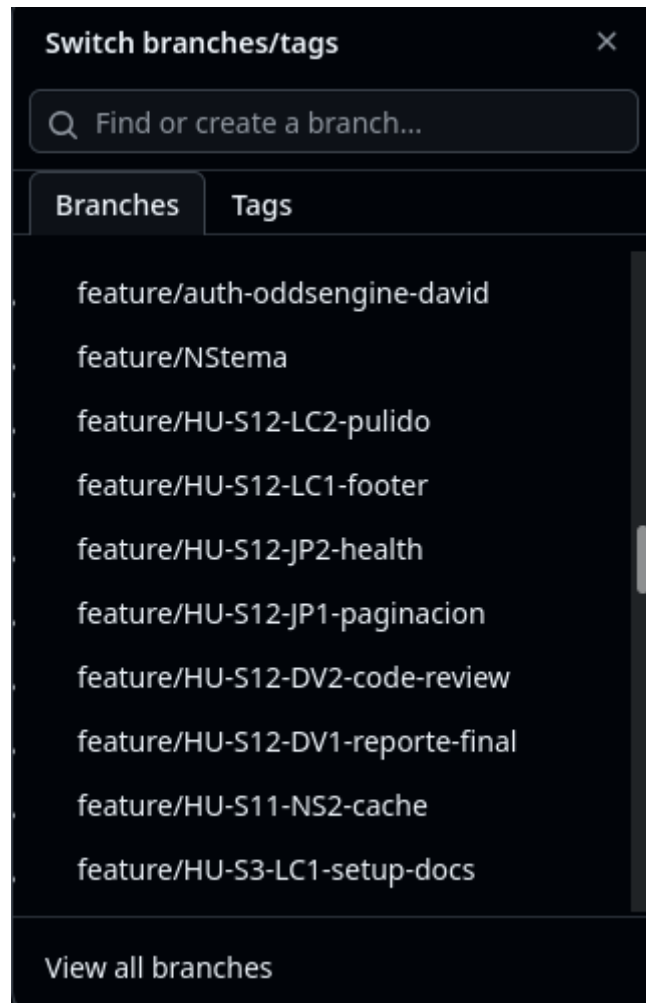


Figura 26: Ramas del repositorio — Vista 3 (feature/HU-S10, feature/HU-S9, feature/HU-S8).

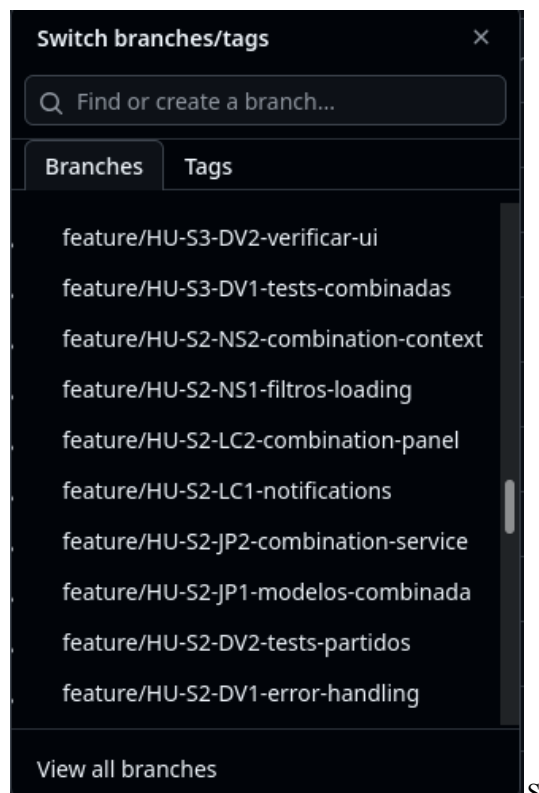
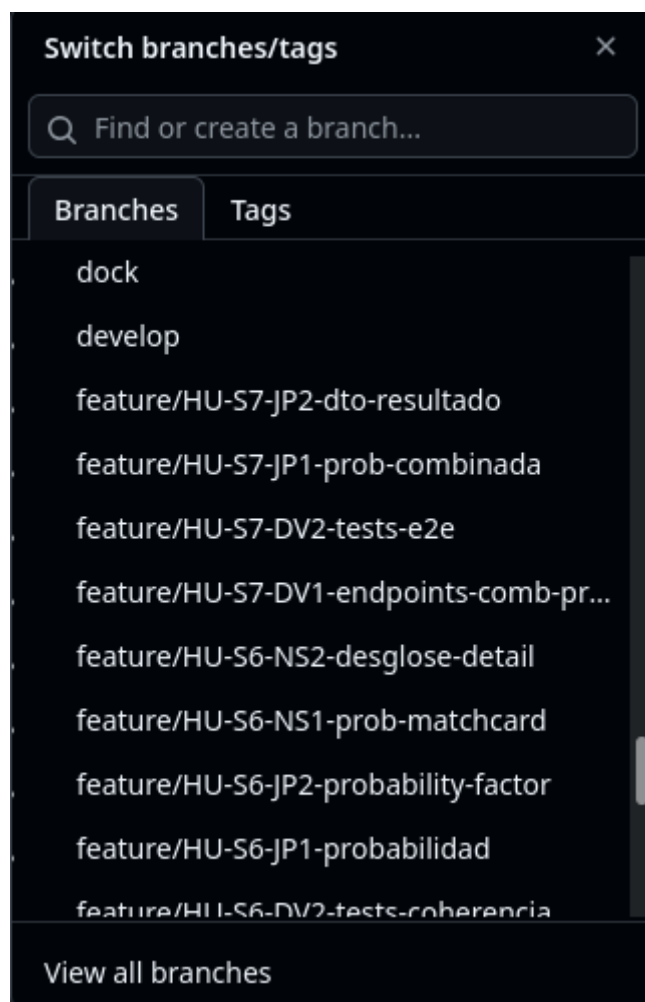
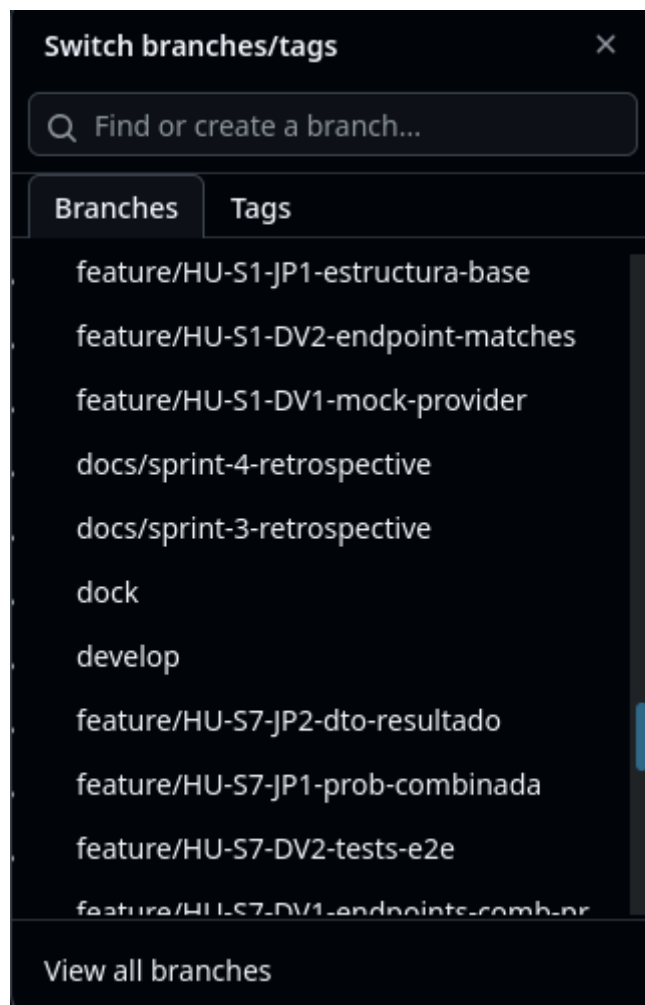
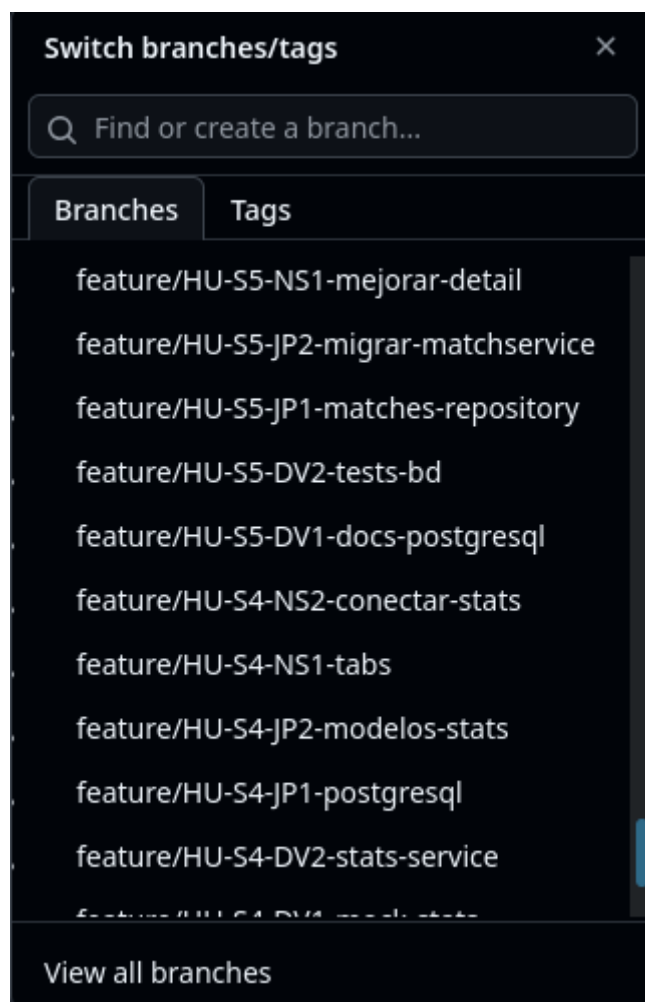
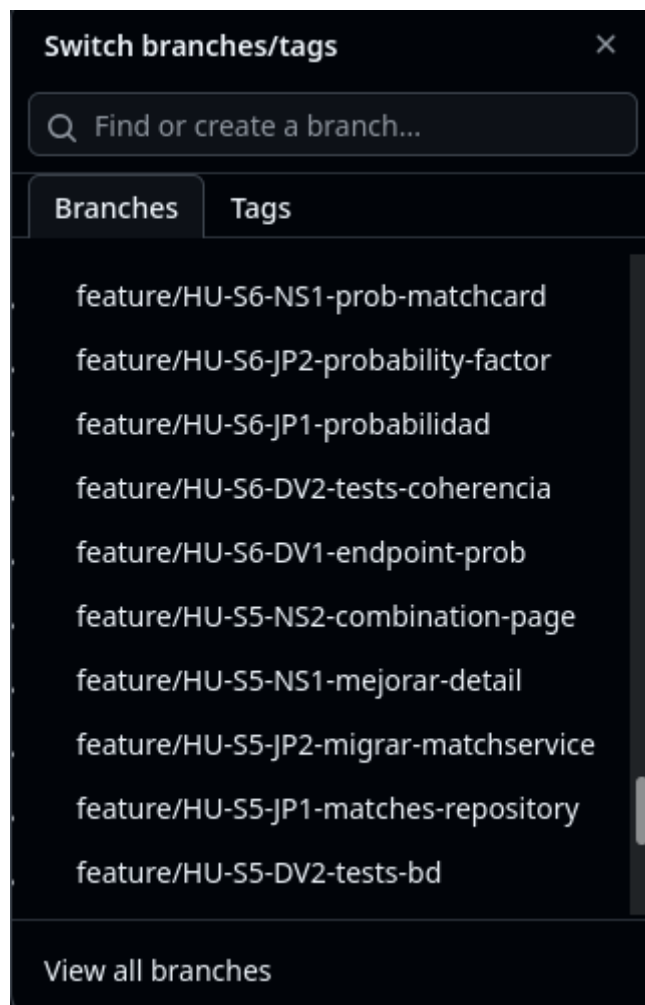
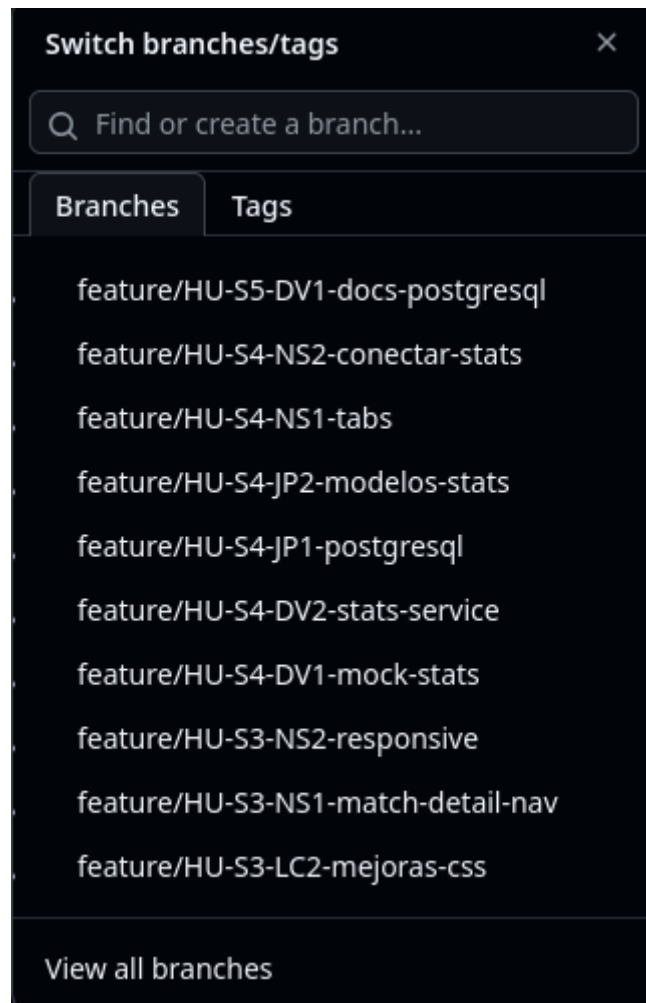


Figura 27: Ramas del repositorio — Vista 4 (feature/auth, feature/metrics, features-david, fix/, funcionalidades-nicolas).







La convención de nombrado de ramas por HU permitió a cada integrante trabajar de forma autónoma sin pisar el trabajo de los demás, y facilitó la revisión de PRs al tener contexto inmediato de qué funcionalidad afectaba cada cambio. Las ramas docs/sprint-X-retrospective evidencian además la documentación continua del proceso a lo largo del proyecto.

Además de la ventajas previamente mencionadas, al tener las branches distribuidas de esta forma el trabajo se reparte de forma optima y se consigue una armonía grupal llevando a cabo los objetivos del proyecto.

5. Análisis profundo: retrospectivas, causas raíz y acciones de mejora

Durante el desarrollo de OddsEngine, se realizaron retrospectivas al finalizar cada sprint, documentadas como issues en GitHub con el label Sprint. El formato utilizado fue: ¿Qué funcionó bien? / ¿Qué podría haber salido mejor? / ¿Qué vamos a mejorar para el próximo sprint?, permitiendo reflexión individual y colectiva sobre el proceso.

Se realizaron retrospectivas desde el Sprint 3 hasta el Sprint 11, totalizando 9 retrospectivas documentadas en el repositorio.

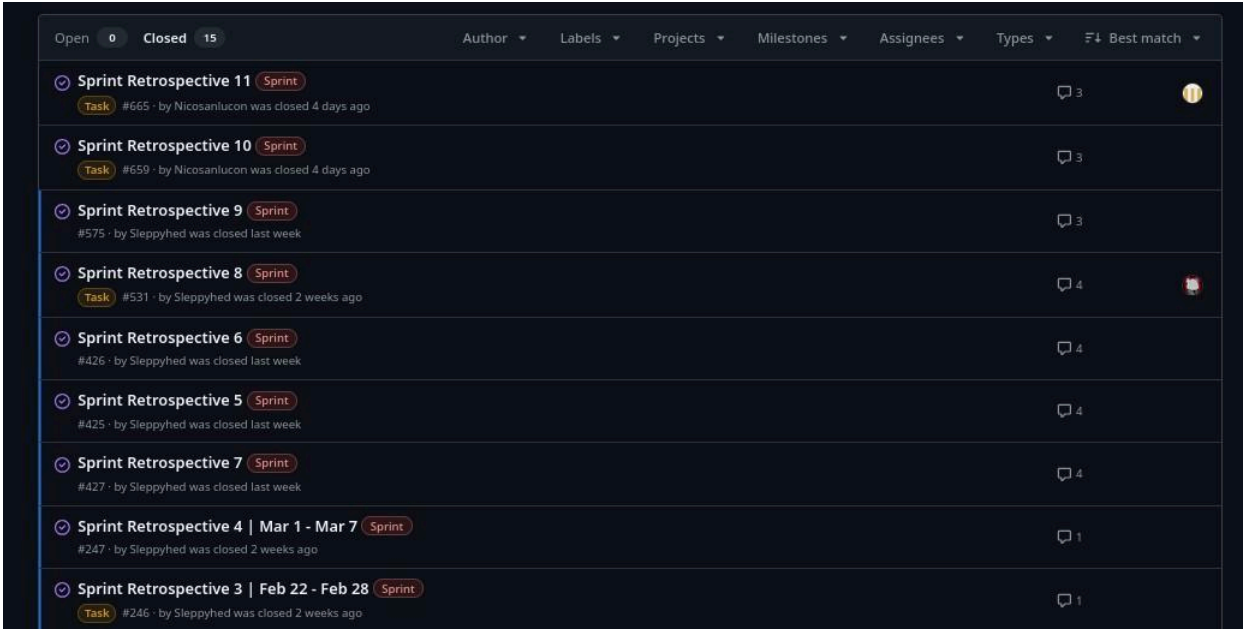


Figura 28: Lista de Sprint Retrospectives documentadas en GitHub Issues.

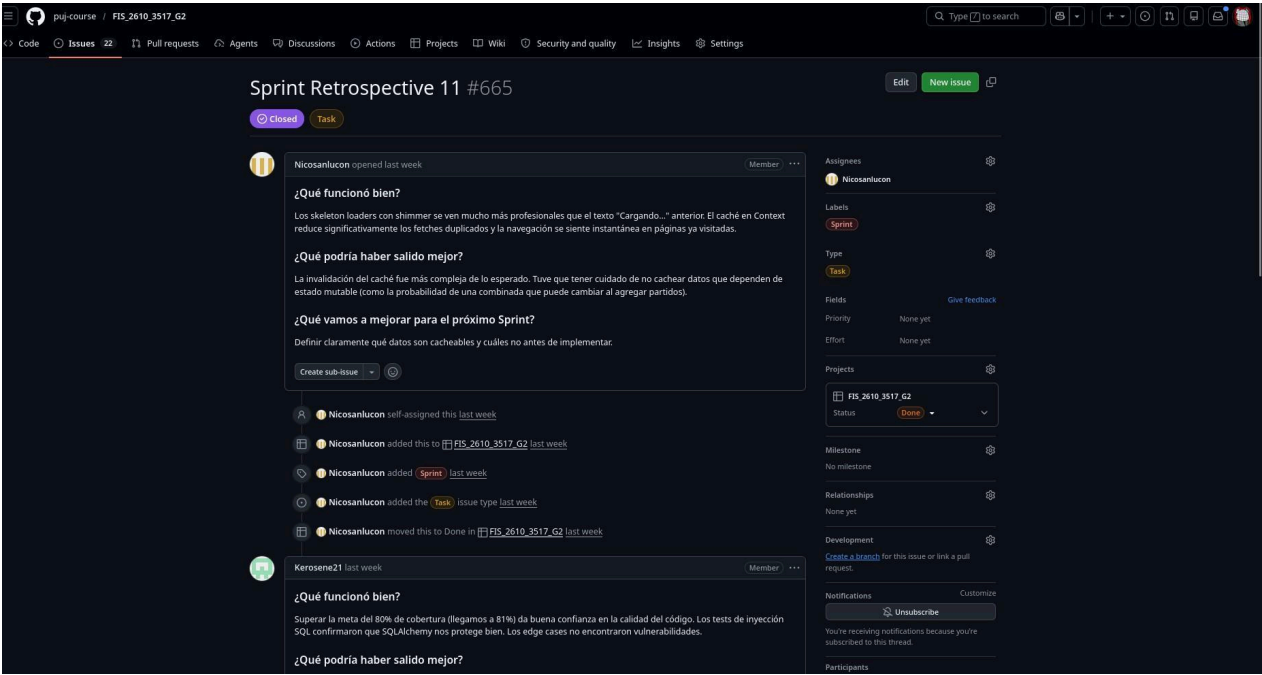


Figura 29: Sprint Retrospective 11 — Ejemplo de formato con reflexiones individuales por integrante.

Principales hallazgos de las retrospectivas

Sprint	¿Qué funcionó bien?	¿Qué mejorar?	Acción tomada
3	Comunicación del equipo mejor	Tiempo de estimación de tareas	Planning más detallado en S4
4	Organización de ramas por HU	Conflictos de merge frecuentes	Política de merge más estricta
5	Migración a PostgreSQL exitosa	Tests insuficientes en migration	Se añadieron tests en S6
6	Fórmula de probabilidad función	técnica acumulada	Refactoring en S7
7	Página CombinationResult com	manejo de test	Tests desde inicio S8
8	Migración completa a PostgreS	delay entre endpoints	Consolidación en todos endpoints S9
9	Historial y exportar implementado	aún por debajo del 80%	Plan de testing en S10 y S11
10	72+ tests pasando, seed unifica	documentación técnica incompleta	Documentación exhaustiva en S11
11	81% coverage alcanzado		Definir límites de caché desde diseño

Tabla 6: Resumen de retrospectivas por sprint — OddsEngine.

Causas raíz identificadas y lecciones aprendidas

A lo largo del proyecto se identificaron de forma recurrente las siguientes causas raíz en los principales problemas del equipo:

Subestimación de la complejidad técnica: La migración a PostgreSQL y la implementación del motor de probabilidad tomaron más tiempo del estimado, afectando los sprints 4 y 5. La acción correctiva fue incluir spikes técnicos al inicio de las fases complejas.

Deuda técnica acumulada: La velocidad de desarrollo inicial sin suficientes tests generó una deuda que debió pagarse en los últimos sprints. La lección fue incorporar testing desde el primer día de cada HU.

Conflictos de merge: Trabajar en ramas de larga duración causó conflictos al integrar. La solución fue reducir el tamaño de las ramas (una por HU) y hacer merge frecuente con main.

Coverage insuficiente: La meta del 80% de cobertura no se planificó desde el inicio, lo que requirió esfuerzo concentrado en los últimos sprints. La acción fue definir metas de coverage por sprint desde el inicio del proyecto.

Documentación tardía: La documentación técnica de endpoints y configuración se dejó para los últimos sprints. La mejora fue definir documentación como criterio de aceptación de cada HU.

Resultados finales del proyecto

Indicador	Resultado
Total de commits en main	670+

Total de HU completadas	68 / 72
Cobertura de tests alcanzada	81%
Milestones completados al 100%	10 / 11
Quality Gate SonarCloud	Pasando
Workflows de CI/CD activos	6 (CI, CD, Docker, SonarCloud, Commits, Auto-PR)

Ramas del repositorio	94
Pull Requests mergeados	700+
Stack tecnológico	FastAPI + React/Vite + PostgreSQL + Docker

Tabla 7: Indicadores finales del proyecto OddsEngine.

6. Análisis Técnico y de Proceso

A lo largo del desarrollo de OddsEngine se identificaron cinco causas raíz recurrentes que afectaron la velocidad y calidad del equipo.

La primera fue la subestimación de la complejidad técnica: tareas como la migración a PostgreSQL y la implementación del motor de probabilidad tomaron más tiempo del estimado porque no se realizaron spikes técnicos previos. La acción correctiva fue incluir una fase de exploración técnica al inicio de cada fase compleja.

La segunda fue la deuda técnica por falta de tests tempranos: el equipo priorizó funcionalidad sobre cobertura en los primeros sprints, lo que generó una deuda que debió pagarse concentradamente en los sprints 10 y 11. La lección fue incorporar tests desde el primer día de cada HU como criterio de aceptación.

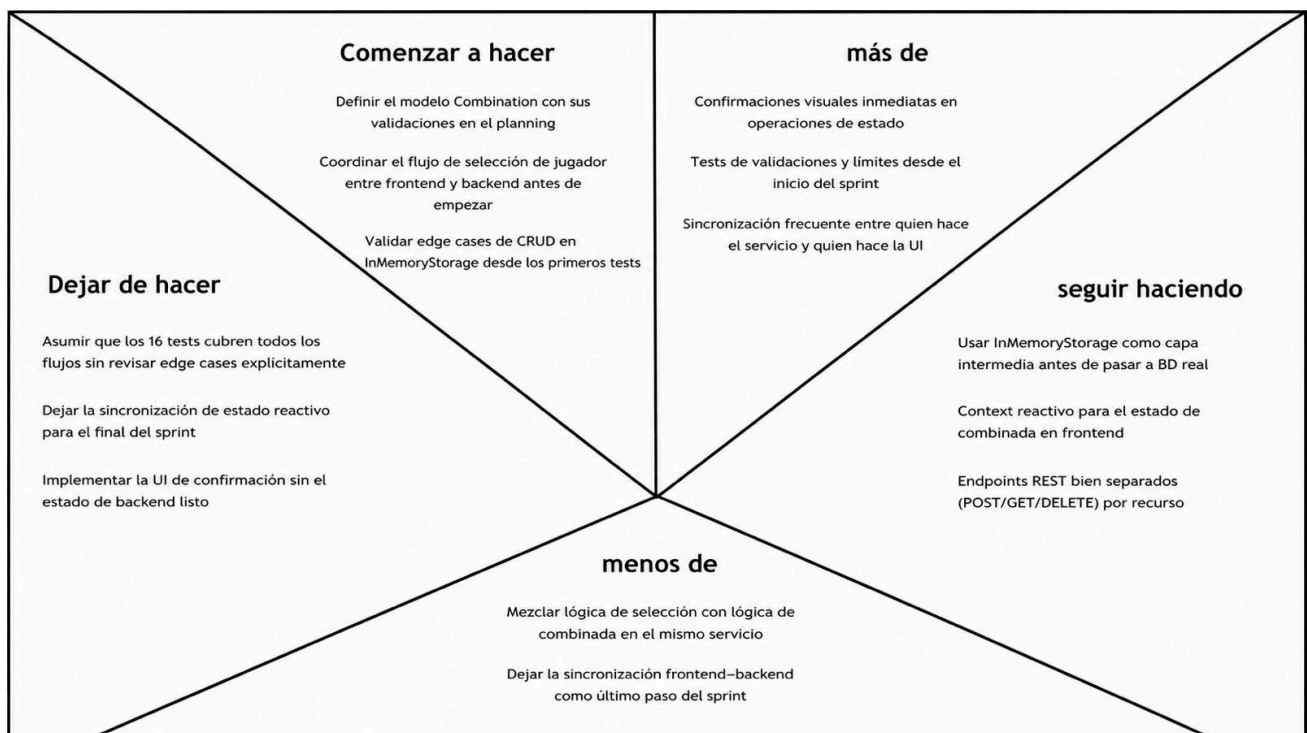
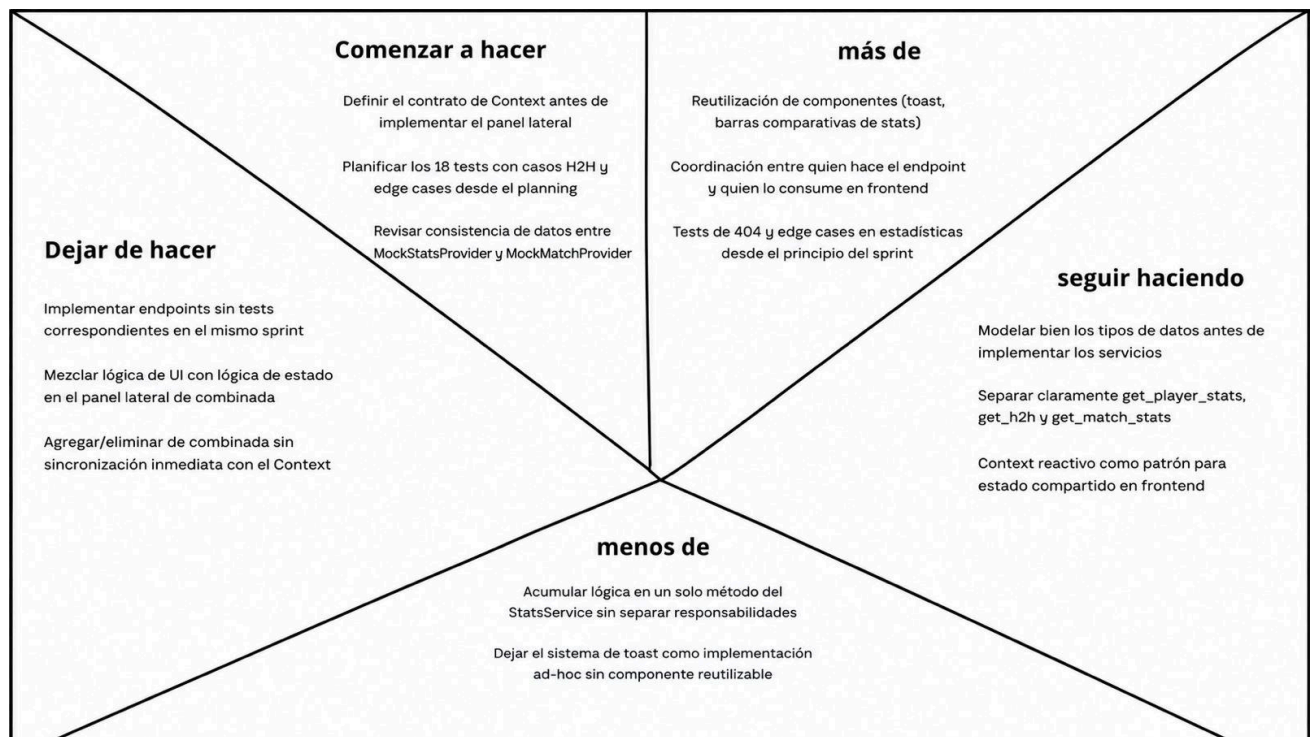
La tercera fue los conflictos de merge frecuentes: trabajar en ramas de larga duración sin sincronización frecuente con develop generaba conflictos al integrar. La solución fue reducir el alcance de las ramas a una por HU y hacer merges más frecuentes.

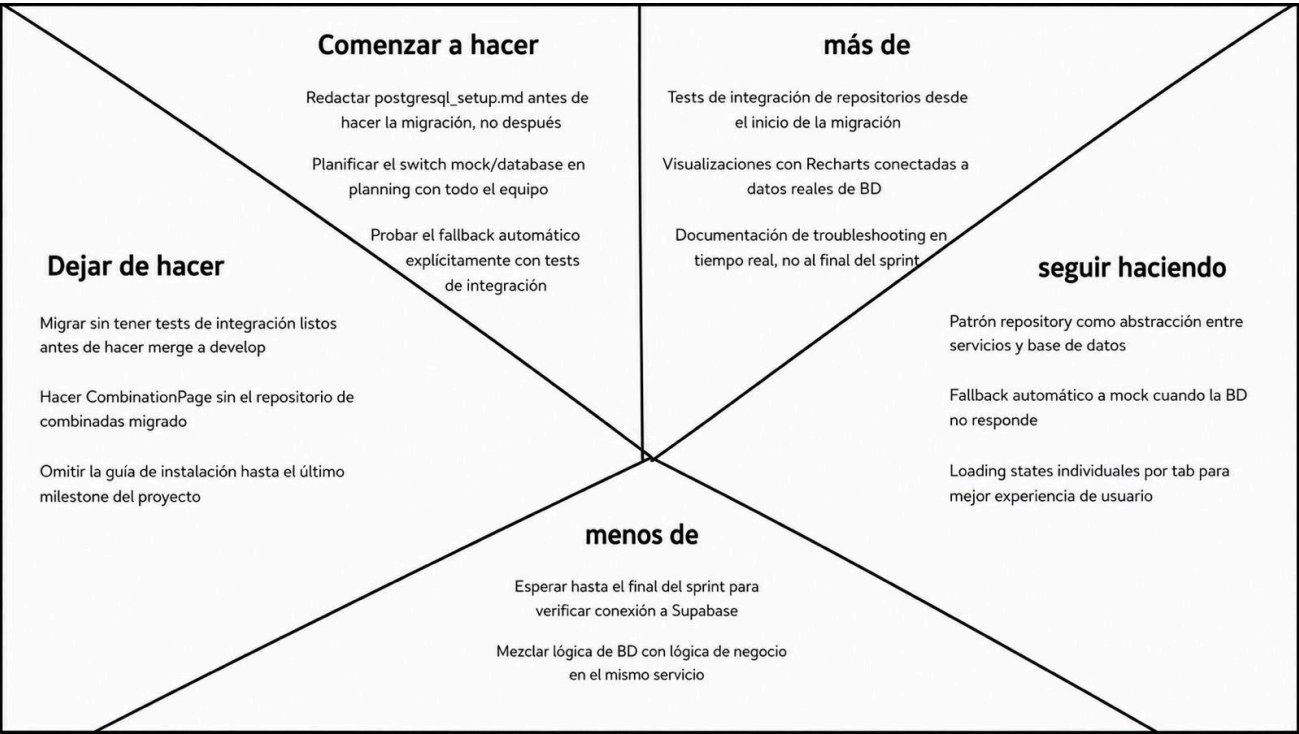
La cuarta fue la cobertura planificada tardíamente: la meta del 80% no se definió como requisito desde el inicio, obligando al equipo a dedicar sprints completos exclusivamente a testing. La mejora fue definir metas de cobertura por sprint desde el planning inicial.

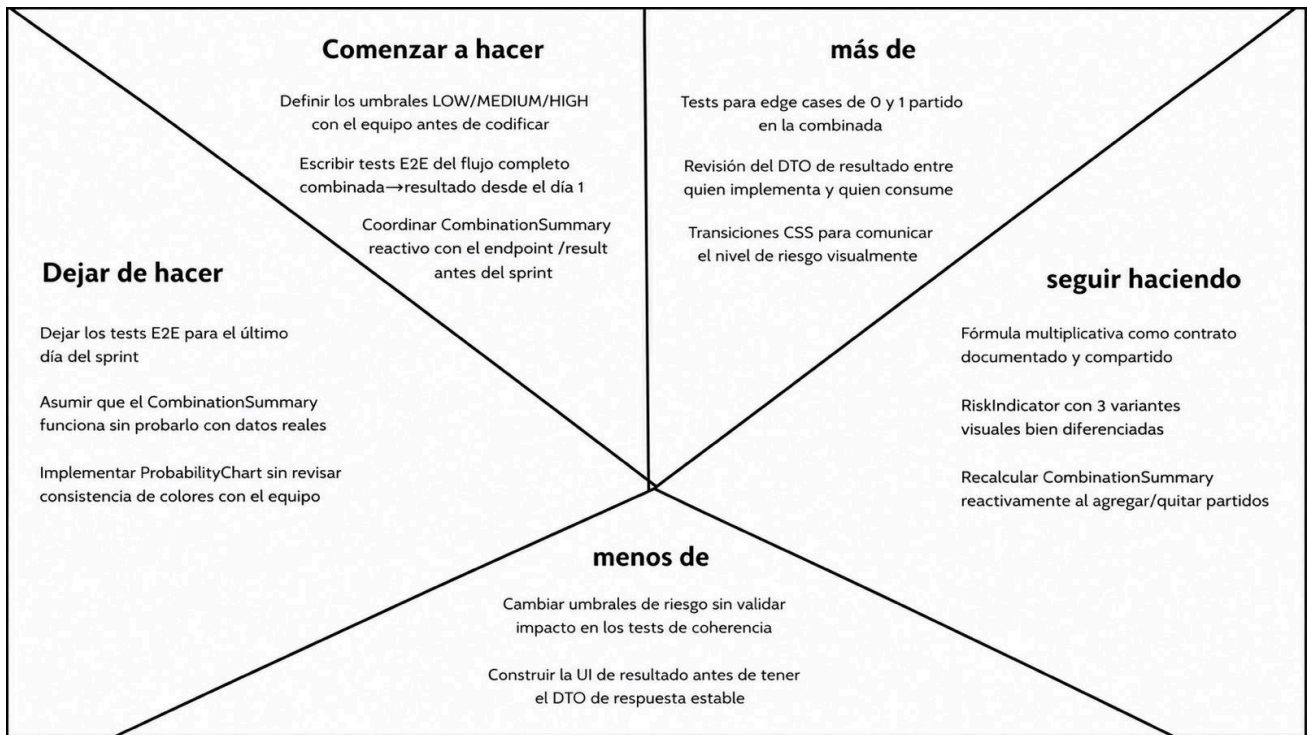
La quinta fue la documentación técnica postergada: endpoints, guías de instalación y configuración de base de datos se documentaron en los últimos sprints, dificultando la incorporación de nuevos miembros y la validación cruzada. La acción fue establecer la documentación como criterio de aceptación obligatorio en cada HU.

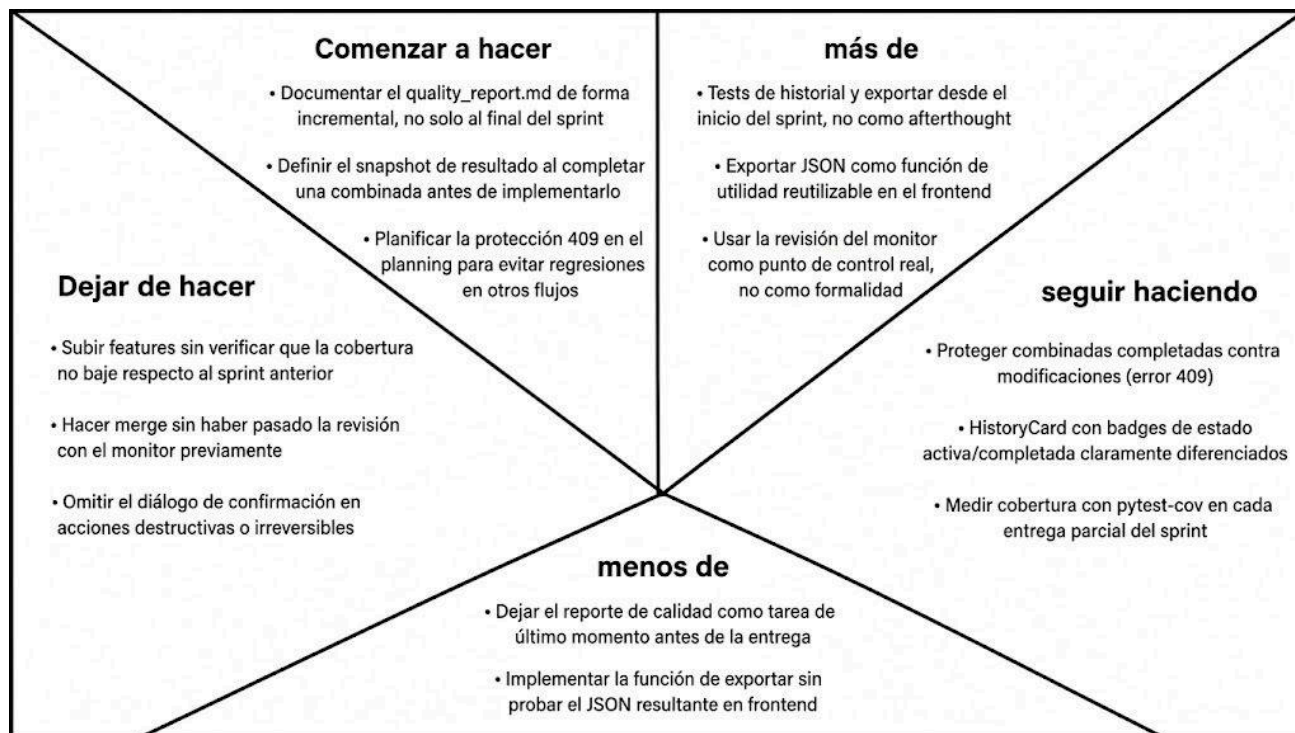
En conjunto, estas lecciones aprendidas llevaron al equipo a cerrar el proyecto con 81% de cobertura, Quality Gate de SonarCloud en verde, 6 workflows de CI/CD activos y un sistema completamente funcional desplegado con Docker, lo que refleja una mejora continua sostenida a lo largo de los 12 sprints.











OddsEngine fue desarrollado en 12 sprints por David Orjuela, Juan Pablo Álvarez, Nicolás Sánchez y Lucas Rincón, alcanzando un cumplimiento del ~99% con 10 de 11 milestones completados al 100%, 68 de 72 historias de usuario cerradas y 81% de cobertura de tests. El equipo gestionó más de 670 commits, 700+ pull requests y 94 ramas bajo una convención clara de nombrado por HU, garantizando trazabilidad y trabajo paralelo sin interferencias. Las principales lecciones aprendidas fueron incorporar testing desde el inicio de cada HU, definir criterios de aceptación antes de codificar y hacer merges frecuentes para evitar conflictos, aspectos que el equipo fue corrigiendo progresivamente sprint a sprint mediante retrospectivas documentadas en GitHub.